



T.C.
İSTANBUL TOPKAPI
ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA
BİLİMLERİ FAKÜLTESİ
YAZILIM MÜHENDİSLİĞİ
BÖLÜMÜ

Android İşletim Sistemi Yapay Zekâ
Asistanı (Android AI Assistant)

Muhammed Chreiki
22040301135

BİTİRME PROJESİ

İSTANBUL 2026

T.C.
İSTANBUL TOPKAPI
ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA
BİLİMLERİ FAKÜLTESİ
YAZILIM MÜHENDİSLİĞİ
BÖLÜMÜ

Android İşletim Sistemi Yapay Zekâ
Asistanı (Android AI Assistant)

Muhammed Chreiki
22040301135

Projenin Danışmanı : Dr. Öğr. Üyesi Bahattin Selim Pamukcu
Jüri Üyesi :
Jüri Üyesi :

AKADEMİK DÜRÜSTLÜK BEYANI

Mezuniyet projesi olarak sunduğum “Android İşletim Sistemi Yapay Zekâ Asistanı (Android AI Assistant)” başlıklı çalışmanın, bilimsel ahlak ve geleneklere uygun olarak tarafımdan yazıldığını, yararlandığım eserlerin tamamının kaynaklarda gösterildiğini ve çalışmam içerisinde kullanıldıkları her yerde atıf yapıldığını belirtir ve onurumla doğrularım.

29.06.2026

Muhammed Chreiki

TEŞEKKÜR

Çalışmalarım süresince bilimsel katkıları ile beni aydınlatan, farklı bakış açıları ile yol gösteren, yakın ilgi ve yardımları ile beni motive eden ve bu günlere gelmemde en büyük katkı sahibi olan sayın Dr. Öğr. Üyesi Bahattin Selim Pamukcu'ya teşekkürü bir borç bilirim.

Deneyisel çalışmalarım sırasında karşılaştığım zorlukları aşmamda yardımlarından dolayı Dr. Öğr. Üyesi Sajjad Nematzadeh Miandoab'a teşekkür ederim.

Ayrıca, çalışmalarım süresince sabır göstererek beni daima destekleyen değerli aileme ve arkadaşlarıma en içten teşekkürlerimi sunarım.

29.06.2026

Muhammed Chreiki

ÖZET

Günümüzde akıllı telefonlarla etkileşim günlük yaşamın vazgeçilmez bir parçası haline gelmiş durumdadır. Ancak bazı senaryolarda telefon ile dokunmatik olarak etkileşim kurmamız zor bir hale gelebilir. Özellikle hareket kısıtı bulunan bireylerin telefonu kullanmaları zorlayıcı olabilir. Bu proje kapsamında, Android işletim sistemine sahip olan akıllı cep telefonlarına yönelik bir yapay zekâ asistanı sistemi geliştirilmiştir.

Bu sistem, kullanıcıların sesli komutlar aracılığıyla telefonlarıyla etkileşim kurmasına olanak sağlamaktadır. Uygulama açma, metin alanlarına yazı yazma, kaydırma, arama yapma ve fotoğraf çekme gibi işlevlerin sesli komutlarla gerçekleştirilmesi hedeflenmiştir. Böylece telefonla doğrudan dokunmatik etkileşim kurmaya duyulan ihtiyaç azaltılarak, ellerin kullanımının kısıtlandığı durumlarda ve hareket kısıtı bulunan kullanıcılar için cihazın sesle kontrol edilebilmesine katkı sağlanmaktadır. Bu projede çoklu dil desteği (İngilizce, Türkçe ve Arapça) sunulmuştur. Önceki yıllara göre Android sürümlerine daha uyumlu ve yenilikçi bir sistem tasarlanmıştır.

Bu çalışmada yöntem olarak birçok modül ve katman ile çözüme gidilmiştir. Bu modüller sistemin ana mimarisini oluşturmaktadır. Kullanıcıdan alınan sesli komutlar STT, NLP, komut sınıflandırma, Android Accessibility Service, backend API, validation, database/cache ve Android execution modüllerinden geçirilerek komutların doğru bir şekilde gerçekleştirilmesi amaçlanmıştır.

Anahtar kelimeler: Android, Sesli komutlar, Makine öğrenmesi

ABSTRACT

Today, interaction with smartphones has become an indispensable part of daily life. However, in certain scenarios, interacting with a phone through touch can become difficult. In particular, using a phone may be challenging for individuals with limited mobility. Within the scope of this project, an artificial intelligence assistant system was developed for smartphones running the Android operating system.

This system enables users to interact with their phones through voice commands. In general, it was aimed to perform functions such as opening applications, typing into text fields, scrolling, searching, and taking photos through voice commands. In this way, the phone can be controlled by voice without requiring direct touch-based interaction. This provides advantages in scenarios such as situations where the use of hands is restricted, and accessibility-related cases where voice-based phone control is needed. In this project, multilingual support for English, Turkish, and Arabic is provided. Compared to earlier approaches, a more up-to-date and innovative system compatible with modern Android versions was designed.

In this study, the solution is built through multiple modules and layers. These modules form the main architecture of the system. Voice commands received from the user are processed through STT, NLP, command classification, Android Accessibility Service, backend API, validation, database/cache, and Android execution modules in order to execute the intended commands correctly.

Keywords: Android, Voice commands, Machine Learning

İÇİNDEKİLER

Sayfa No

AKADEMİK DÜRÜSTLÜK BEYANI	i
TEŞEKKÜR	ii
ÖZET	iii
ABSTRACT	iv
İÇİNDEKİLER	v
ŞEKİLLER DİZİNİ	vii
ÇİZELGELER DİZİNİ	viii
KISALTMALAR	ix
1. GİRİŞ	1
1.1. Problem Tanımı	1
1.2. Projenin Amacı ve Kapsamı	2
1.3. Hedef Kitle	2
1.4. Neden Android İşletim Sistemi?	2
2. KAYNAK ARAŞTIRMASI	3
3. KISITLAR	5
3.1. Gerçekçi Kısıtlar	5
3.2. Risk Analizi	5
4. MATERYAL VE YÖNTEM	6
4.1. Materyal	6
4.1.1. Veri Seti	6
4.1.2. Yazılım ve Geliştirme Ortamı	8
4.1.3. Android Uyumluluğu ve Sistem Gereksinimleri	8
4.2. Yöntem	9
4.2.1. Sesli Komutların Metne Dönüştürülmesi (STT)	9
4.2.2. Metin Ön İşleme, Normalizasyon ve Kural Tabanlı Çözümleme (Backend)	10
4.2.3. Komut Sınıflandırma (Makine Öğrenmesi)	11
4.2.4. Intent Doğrulama ve Parametre Çıkarımı	12
4.2.5. Model Servisleme ve Backend API (FastAPI)	13
4.2.6. Veritabanı ve Cache	15
4.2.7. Android Erişilebilirlik Servisi ile Arayüz Etkileşimi	16
4.3. Gelişmiş Komut Etkileşimleri	21
4.3.1. Dinamik Kullanıcı Arayüzü Etkileşimi	21
4.3.2. Özel Komut Akışları	23
5. SONUÇLAR	25
5.1. Gereksinim Listesi	25
5.1.1. Fonksiyonel Gereksinimler	25
5.1.2. Fonksiyonel Olmayan Gereksinimler	26
5.1.3. Desteklenen Komut Kategorileri	27

5.2.	Sistem Akış Diagramları	28
5.2.1.	Use Case Diagram	28
5.2.2.	Class Diagram	29
5.2.3.	Deployment Diagram	30
5.2.4.	Sequence Diagram.....	31
5.3.	Makine Öğrenmesi Modelinin Sonuç Metrikleri	32
5.4.	Mobil Uygulama Kullanıcı Arayüzleri	32
5.4.1.	AI Asistan Ana Ekranı.....	32
5.4.2.	İzin Yönetimi Ekranı	34
5.4.3.	Özel Komutlar Ekranı.....	35
5.4.4.	Sesli Komut Kılavuzu Ekranı	37
5.4.5.	Ayarlar Ekranı	38
5.5.	Uçtan Uca Sistem Sonucu.....	39
6.	TARTIŞMA VE GELECEK ÇALIŞMALAR	40
7.	EKLER	41
7.1.	İş Paketleri.....	41
7.1.1.	İş Paketleri Tablosu ve Çıktıları	41
7.1.2.	İş Paketleri Zamanlama Tablosu	42
7.2.	Projenin Etkisi ve Katkısı.....	42
7.2.1.	Sürdürülebilir Kalkınma Amaçları	42
7.2.2.	SKA Açıklaması.....	43
7.2.3.	SKA Alt Maddeleri.....	43
8.	KAYNAKLAR	44
9.	ÖZGEÇMİŞ.....	45

ŞEKİLLER DİZİNİ

Sayfa No

Şekil 1. Veri seti İngilizce komut örnekleri.....	7
Şekil 2. Veri seti Türkçe komut örnekleri	7
Şekil 3. Veri seti Arapça komut örnekleri	8
Şekil 4. Sesli komut işleme katmanları.....	9
Şekil 5. STT aşamalarının temel ilerleme aşamaları (<i>Speech to Text Guide: Convert Audio to Text 3x Faster (2025)</i> , n.d.)	10
Şekil 6. XLM-RoBERTa tabanlı niyet sınıflandırma pipeline'ı	12
Şekil 7. Backend FastAPI endpoint'leri	14
Şekil 8. Veritabanı ERD Diagramı	16
Şekil 9. Android Komut yürütme akışı.....	17
Şekil 10. Android erişilebilirlik (Numaralandırma örneği)	19
Şekil 11. Android erişilebilirlik (Grid örneği).....	20
Şekil 12. Chrome'da aday numaralandırma örneği.....	21
Şekil 13. Uygulama açma akışında aday seçtirme örneği	22
Şekil 14. Yazı girme akışında aday input seçtirme örneği	23
Şekil 15. Sistemin genel use case diagramı	29
Şekil 16. Sistem mimarisini içeren class diagramı	30
Şekil 17. Sistem bileşenlerinin deployment diagramı	30
Şekil 18. İş akışı ve sistem bileşenleri arasındaki iletişimi ele alan sequence diagramı	31
Şekil 19. Uygulama içerisinde ana sayfa	33
Şekil 20. AI Asistan ekran üstü komut bekleme arayüzü.....	33
Şekil 21. Uygulama içerisinde izin yönetme sayfası	35
Şekil 22. Uygulama içerisinde özel komut işleme sayfası.....	36
Şekil 23. Uygulama içerisinde kullanıcı yardım kılavuzu sayfası.....	37
Şekil 24. Uygulama içerisinde ayarlar sayfası.....	38

ÇİZELGELER DİZİNİ

Sayfa No

Tablo 1. Karşılaşılabilecek sorunlar ve doğabilecek riskler tablosu.....	5
Tablo 2. Fonksiyonel gereksinimler listesi.....	25
Tablo 3. Fonksiyonel olmayan gereksinimler listesi	26
Tablo 4. Sistem Tarafından Desteklenen Komut Kategorileri	27
Tablo 5. Model sonuç metrikleri	32
Tablo 6. Uçtan Uca İşlev Testleri	39
Tablo 7. İş paketleri, çıktıları ve projede etkisi	41
Tablo 8. İş zamanlama planlaması.....	42
Tablo 9. Sürdürülebilir Kalkınma Amaçları	42

KISALTMALAR

STT	: Speech-To-Text (Konuşmadan Metne Çevirme)
NLP	: Natural Language Processing (Doğal Dil İşleme)
UI	: User Interface (Kullanıcı Arayüzü)
iOS	: iPhone Operating System (iPhone İşletim Sistemi)
API	: Application Programming Interface (Uygulama Programlama Arayüzü)
TF-IDF	: Term Frequency – Inverse Document Frequency (Terim Frekansı – Ters Belge Frekansı)
FG	: Fonksiyonel Gereksinimler (Functional Requirements)
FOG	: Fonksiyonel Olmayan Gereksinimler (Non Functional Requirements)
ERD	: Entity Relationship Diagram (Varlık İlişki Diagramı)
SKA	: Sürdürülebilir Kalkınma Amaçları
HTTP	: Hyper Text Transfer Protocol (Hiper Metin Aktarım Protokolü)
JSON	: Javascript Object Notation (JavaScript Nesne Gösterimi)

1.GİRİŞ

Bu bitirme projesinde, Android işletim sisteminde sesli komutlar aracılığı ile belirli işlemleri yerine getiren bir yapay zekâ asistanı geliştirilmiştir. Projenin gereksinimleri gereği komutların analiz edilmesi ve doğru bir şekilde gerçekleştirilmesi için STT, NLP, makine öğrenmesi gibi yöntemlerden yararlanılmıştır.

Bu projenin önemi özellikle cep telefonlarıyla etkileşimin fiziksel olarak zor olduğu, hareket engelliliği durumlarında kullanımı kolaylaştırmaya katkı sağlayabilir. İlerleyen kısımlarda kullanılan yöntemler, sistem mimarisi ve uygulama süreçleri detaylıca açıklanmıştır.

1.1. PROBLEM TANIMI

Günümüzde, özellikle 21. yüzyıl sonrası cep telefonlarıyla etkileşimimiz kaçınılmaz bir ihtiyaç niteliğindedir. Artık herkesin akıllı bir cep telefonuna sahip olması temel bir ihtiyaç haline gelmiş durumdadır. Ancak bazı durumlarda, örneğin yemek pişirme, fiziksel aktivite, ellerin meşgul olması gibi senaryolarda, telefon ile doğrudan dokunmatik bir şekilde etkileşime geçmek zor bir hale gelebilmektedir. Bunun yanı sıra hareket yönünden kısıtlı olan bireylerin telefon ile etkileşmesi sınırlayıcı ve zorlayıcı bir hale gelmektedir.

Bir diğer önemli nokta da şudur, bir programın açılması veya UI bileşeninin manuel olarak seçilmesi, görsel odaklanma ve hassas dokunma gerektirir. Ayrıca programların konumlarının değişken olması veya bazı UI bileşenlerinin ekranın köşelerinde bulunması, özellikle tek elle mobil cihaz ile etkileşim senaryolarında zorlaştırabilmektedir (Zhong et al., 2014).

1.2. PROJENİN AMACI VE KAPSAMI

Bu projenin amacı, Android işletim sisteminde kullanıcıdan alınan sesli komutları eylem haline dönüştürüp uygulayan bir yapay zekâ sistemi tasarlamak ve geliştirmektir. Sistem, doğrudan dokunmatik etkileşime ihtiyaç duymadan kullanıcı arayüzü bileşenleriyle sesli komutlar aracılığıyla etkileşim kurulmasını hedeflemektedir..

Bunun yanı sıra elleri kullanma açısından kısıt oluşturan senaryolarda, mobil telefonun sesli olarak kontrol edilmesinin sağlanması amaçlanmaktadır. Bazı durumlarda da bir elimizin meşgul olduğu durumlarda sesli komutların gerçekleştirilmesi sayesinde dokunma ile etkileşime duyulan ihtiyacın azaltılmasına katkı sağlayabileceği düşünülmektedir.

1.3. HEDEF KİTLE

Projenin öncelikli hedef kitlesi hareket kısıtı olan bireylere mobil cihaz kullanımı açısından kolaylık sağlamaktır. Bunun dışında genel kullanıcı olarak yetişkin olmak üzere herkes kullanılabilir.

Hareket kısıtı bulunan veya telefonu elleriyle kullanmakta zorlanan kullanıcılar için uygulamanın ilk kurulumu, gerekli Android izinlerinin verilmesi ve erişilebilirlik servisinin etkinleştirilmesi aşamalarında bir yardımcı kişiden destek alınması gerekebilir. Bu durum, sistemin günlük kullanım amacını ortadan kaldırmamakta; Android işletim sistemlerinin izin ve güvenlik yapısından kaynaklanan ilk yapılandırma gereksinimi olarak değerlendirilmektedir. Gerekli izinler verildikten ve asistan etkinleştirildikten sonra kullanıcı, desteklenen işlemlerin büyük bir bölümünü sesli komutlar aracılığıyla gerçekleştirebilir.

1.4. NEDEN ANDROID İŞLETİM SİSTEMİ?

Bu çalışmada sadece Android işletim sisteminin seçilmesinin birçok nedeni vardır. Android işletim sisteminin en büyük avantajı erişilebilirlik servisine (Accessibility Service) sahip olmasıdır. Erişilebilirlik servisi aracılığıyla programlarda kullanıcı arayüzü (UI) bileşenlerine erişme imkanı sunar.

Bu sayede programın kendisinden bağımsız olarak kullanıcı arayüzü etkileşimlerinin gerçekleştirilebilmesi mümkündür (*AccessibilityService | API Reference | Android Developers*, n.d.).

Ayrıca Android işletim sisteminin açık kaynaklı yapısı, sistem seviyesinde etkileşim gerektiren uygulamaların geliştirilmesine olanak sağlamaktadır (*Android Open Source Project*, n.d.).

StatCounter Global Stats verilerine göre Android, Mayıs 2026 itibarıyla dünya genelindeki mobil işletim sistemi kullanımının yaklaşık %68'ini oluşturarak en yüksek kullanım payına sahip mobil işletim sistemi konumundadır (*Mobile Operating System Market Share Worldwide | Statcounter Global Stats*, n.d.).

iOS işletim sistemiyle karşılaştıracak olursak, iOS'te uygulamaların sandbox yapısından dolayı güvenlik ve gizlilik politikaları doğrultusunda üçüncü taraf uygulamaların kullanıcı arayüzü bileşenlerine sistem genelinde erişilebilirlik açısından sınırlanmaktadır (*Accessibility | Apple Developer Documentation*, n.d.; *App Sandbox | Apple Developer Documentation*, n.d.; *Security of Runtime Process in IOS, IPadOS, and VisionOS - Apple Support*, n.d.). Bu durum, sesli komutları uygulamadan bağımsız bir şekilde kullanıcı arayüzü etkileşimi için ek geliştirme gereksinimleri ortaya koymaktadır. Bu nedenlerden dolayı çalışmanın Android işletim sistemi doğrultusunda uygulanması daha uygun görülmüştür.

2.KAYNAK ARAŞTIRMASI

Uzun bir literatür taraması ve kaynakların araştırılması sonrası yapılmış olan bu çalışmaya benzerlik gösteren ve tamamlanmış olan birkaç proje incelenmiştir.

Voice Access, Google tarafından geliştirilen ve milyonlarca kişinin indirdiği bir uygulamadır. Bu uygulama sesli komutlar ile kullanıcı arayüzü (UI) bileşenlerinin kontrol edilebildiği ve Google Play Store'da bulunuyor (*Use Voice Access to Control Your Android Device with Your Voice*, n.d.; *Voice Access - Apps on Google Play*, n.d.).

Bu uygulamanın kullanıcı geri dönüşleri ve kendi deneyimime dayalı olarak bu uygulamanın kritik sorunlar içerdği görülmüştür. Bu sorunların en öne çıkanları komut algılama konusunda tutarsız olması, komutu doğru bir şekilde alamaması ve komut gerçekleştirme konusunda esnek bir yapıya sahip olmadığına varılmıştır. Bu çalışmada söz konusu sorunlardan kaçınmaya ve esnek bir sistemin tasarlanmasına özen gösterilmiştir.

Justspeak, 2014 yılında bu çalışmayla benzer amaçlar doğrultusunda geliştirilmiş olan bir yardımcı asistan projesidir (Zhong et al., 2014). Geçtiğimiz yıllarda akıllı telefonların, yapay zekânın ve teknolojinin gelişimiyle beraber sistem mimarisi, kullanıcı gereksinimleri ve kullanıcı senaryoları farklılık göstermektedir. Bu farklılıklar nedeniyle bu çalışmada söz konusu değişim ve gelişimleri göz önünde bulundurarak güncel Android versiyonlarına uygun bir sistemin ortaya çıkarılması hedeflenmiştir. Araştırmalar sonrasında Justspeak uygulamasının güncel Android sürümleriyle uyumlu olan ve erişilebilen bir sürümüne rastlanmamıştır. Bu durum güncel ve sürdürülebilir bir çözüme olan ihtiyacı ortaya çıkarmaktadır.

Bu çalışmanın literatür çalışmalarına katkısı aşağıdaki gibidir:

- Sesli komutların STT, NLP ve komut sınıflandırma adımlarına ayrıldığı modüler bir sistem tasarlamak.
- Ticari ve kapalı kaynaklı çözümlerden farklı olarak, akademik çalışmalarda yeniden üretilebilir bir sistem yapısını ortaya koymak.
- Literatür çalışmalarından farklı olarak sisteme çoklu dil desteğinin kazandırılması. Bu kapsamda sistem, İngilizce, Türkçe ve Arapça dillerini destekleyecek şekilde tasarlanmıştır. Bu dillerin desteklenmesi literatüre bir katkı niteliğindedir.

3. KISITLAR

3.1. GERÇEKÇİ KISITLAR

• **Zaman:** Bu projenin farklı Android sürümlerine entegre edilebilmesi için esnek bir zaman planlamasına ihtiyaç duyulmaktadır. Dönem süresi dikkate alınarak projenin kapsamı sınırlı tutulmuştur; bu nedenle sistemin tüm Android sürümlerinde sorunsuz çalışmaması muhtemeldir.

• **Teknik kısıtlar:** Android işletim sistemi güvenlik ve gizlilik koşulları nedeniyle sistemin bazı cihaz ve Android sürümlerinde tamamen entegre olmaması gibi sorunların çıkabilmesi.

• **Donanım/Performans:** Bazı eski ve donanımsal olarak zayıf olan cihazlarda STT işleminde hız ile ilgili sorunlar ve performansın düşebilmesi.

• **Bütçe kısıtı:** Gelişmiş AI modeli için ücretli API kullanımı gerekebilir; sınırlı kaynakla optimize edilmiş çözümler geliştirilmiştir.

3.2. RİSK ANALİZİ

Proje sürecinde ortaya çıkabilecek teknik ve operasyonel riskler Tablo 1’de olduğu gibi tanımlanmıştır. Bu risklere karşı çözüm stratejileri geliştirilmiştir. Özellikle sesli komutların doğru algılanması, model doğruluğu ve sistem uyumluluğu en kritik riskler olarak değerlendirilmiştir.

Tablo 1. Karşılaşılabilecek sorunlar ve doğabilecek riskler tablosu

Risk	Olasılık	Etki	Çözüm
Makine öğrenmesi modelinin düşük doğruluk verebilmesi	Düşük	Yüksek	Veri seti artırma, çoğaltma ve model geliştirme yöntemlerini uygulama
Speech-To-Text hatalı komut algılayabilmesi	Orta	Yüksek	doğrulama, kullanıcı geri bildirimi ve komutun yeniden alınması

Farklı Android sürümlerinde uyumsuzlukların oluşabilmesi	Orta	Orta	API uyumluluk kontrollerinin yapılması ve iyileştirilmesi
Çoklu dil desteğinde hataların oluşabilme olasılığı	Orta	Yüksek	Dil bazlı iyileştirme ve ön işleme işlemlerinin gerçekleştirilmesi
Erişim servisinin kısıtları	Düşük	Yüksek	Alternatif kontrol yöntemlerinin bulunması

4. MATERYAL VE YÖNTEM

4.1. MATERYAL

Bu proje kapsamında kullanılan materyaller; yazılım ve donanım araçları, veri setleri ve geliştirme ortamlarından oluşmaktadır.

4.1.1. VERİ SETİ

Bu çalışma kapsamında, İngilizce, Türkçe ve Arapça dillerini içeren çok dilli bir veri seti oluşturulmuştur. Veri seti, uygulama açma, metin yazma, kaydırma, arama yapma gibi farklı kullanıcı senaryolarını kapsayan komutlardan oluşmaktadır. Her intent sınıfı için farklı ifade biçimlerini içeren örnekler eklenerek modelin genelleme yeteneği artırılmıştır. Veri seti, eğitim (train), doğrulama (validation) ve test olarak üçe ayrılmıştır.

Her bir komut için belirli bir cümle dizilimi değil de birden çok örnek olacak şekilde veri seti oluşturulmuştur. Örnek olarak “Open Whatsapp” ve “launch Whatsapp” aynı intent (niyet)’i içerir, aynı işlemi yapması beklenir.

Veri setinde 33 intent ailesi, kategorik parametre birleşimleriyle 58 model etiketi ve backend sözleşmelerinde toplam 41 desteklenen intent vardır. Oluşturulan veri seti ile ilgili Şekil 1, Şekil 2, ve Şekil 3’te olduğu gibi her üç dilden farklı örnekler gösterilmektedir.

text	lang	split	intent	parameters	target_json
open WhatsApp	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Camera	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open YouTube	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Instagram	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Settings	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Maps	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Gallery	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open Chrome	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
open	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch WhatsApp	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Camera	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch YouTube	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Instagram	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Settings	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Maps	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Gallery	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch Chrome	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
launch	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start WhatsApp	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Camera	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start YouTube	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Instagram	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Settings	EN	train	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Maps	EN	val	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Gallery	EN	val	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}
start Chrome	EN	val	OPEN_APP		{"intent": "OPEN_APP", "parameters": {}}

Şekil 1. Veri seti İngilizce komut örnekleri

ön kamerayı aç ve fotoğraf al	TR	test	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
selfie kamerasından fotoğraf çek	TR	test	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
bir çekim yap	TR	test	TAKE_PHOTO		{"intent": "TAKE_PHOTO", "parameters": {}}
ön kamerayla fotoğraf çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
selfie çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kamera ile resim çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
selfie çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kamerayı aç ve fotoğraf çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kameradan fotoğraf çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kamerayla bir resim çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
selfie kamerasıyla fotoğraf çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön taraftan fotoğraf çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kamera fotoğrafı çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
selfie fotoğrafı çek	TR	train	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
arka kamerayla fotoğraf çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamera ile resim çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
ana kamerayla fotoğraf çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamerayı aç ve fotoğraf çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamerayla bir resim çek	TR	val	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
ön kamera ile fotoğraf al	TR	val	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ön kamerayı kullanıp fotoğraf çek	TR	val	TAKE_PHOTO	front	{"intent": "TAKE_PHOTO", "parameters": {"camera": "front"}}
ana kamera ile resim çek	TR	val	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamera fotoğrafı çek	TR	test	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamera ile fotoğraf al	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamerayı kullanıp fotoğraf çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kameradan çekim yap	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka lensle fotoğraf çek	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}
arka kamerayı aç ve fotoğraf al	TR	train	TAKE_PHOTO	back	{"intent": "TAKE_PHOTO", "parameters": {"camera": "back"}}

Şekil 2. Veri seti Türkçe komut örnekleri

الغ كتم صوت الوسائط	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
افتح صوت الميديا من جديد	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
اعد صوت الوسائط	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
ازل كتم مستوى الصوت	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
شغل صوت التشغيل مرة اخرى	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
اجعل صوت الميديا مسموعا	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
الغ كتم صوت التشغيل	AR	train	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
رجع صوت الوسائط	AR	val	ADJUST_VOLUME	unmute	{"intent": "ADJUST_VOLUME", "parameters": {"volume_action": "unmute"}}
اجعل صوت الوسائط منخفضا	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
ضع الصوت على مستوى منخفض	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
اخفض صوت الميديا الى مستوى منخفض	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
اجعل مستوى الصوت منخفضا	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
استخدم صوت ميديا منخفض	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
ضع الصوت على المنخفض	AR	train	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
اجعل صوت التشغيل منخفضا	AR	val	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
حول الصوت الى مستوى منخفض	AR	val	ADJUST_VOLUME	low	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "low"}}
اجعل صوت الوسائط على الحد الاقصى	AR	train	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
ضع الصوت على اعلى مستوى	AR	train	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
ارفع صوت الميديا للآخر	AR	val	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
اجعل مستوى الصوت ماكس	AR	val	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
استخدم اعلی صوت للميديا	AR	test	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
ضع الصوت على الحد الاعلى	AR	test	ADJUST_VOLUME	max	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "max"}}
اجعل صوت الوسائط متوسطا	AR	train	ADJUST_VOLUME	medium	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "medium"}}
ضع الصوت على مستوى متوسط	AR	train	ADJUST_VOLUME	medium	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "medium"}}
اجعل مستوى الصوت في الوسط	AR	train	ADJUST_VOLUME	medium	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "medium"}}
اجعل صوت الميديا متوسطا	AR	train	ADJUST_VOLUME	medium	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "medium"}}
استخدم صوت ميديا متوسط	AR	train	ADJUST_VOLUME	medium	{"intent": "ADJUST_VOLUME", "parameters": {"volume_level": "medium"}}

Şekil 3. Veri seti Arapça komut örnekleri

Veri seti oluşturulması sonrasında NLP kullanarak classification model geliştirme aşamasına geçilmiştir.

4.1.2. YAZILIM VE GELİŞTİRME ORTAMI

Bu projede kullanılan yazılım ve araçlar aşağıdaki gibi listelenmektedir (*Download Android Studio & App Tools - Android Developers*, n.d.).

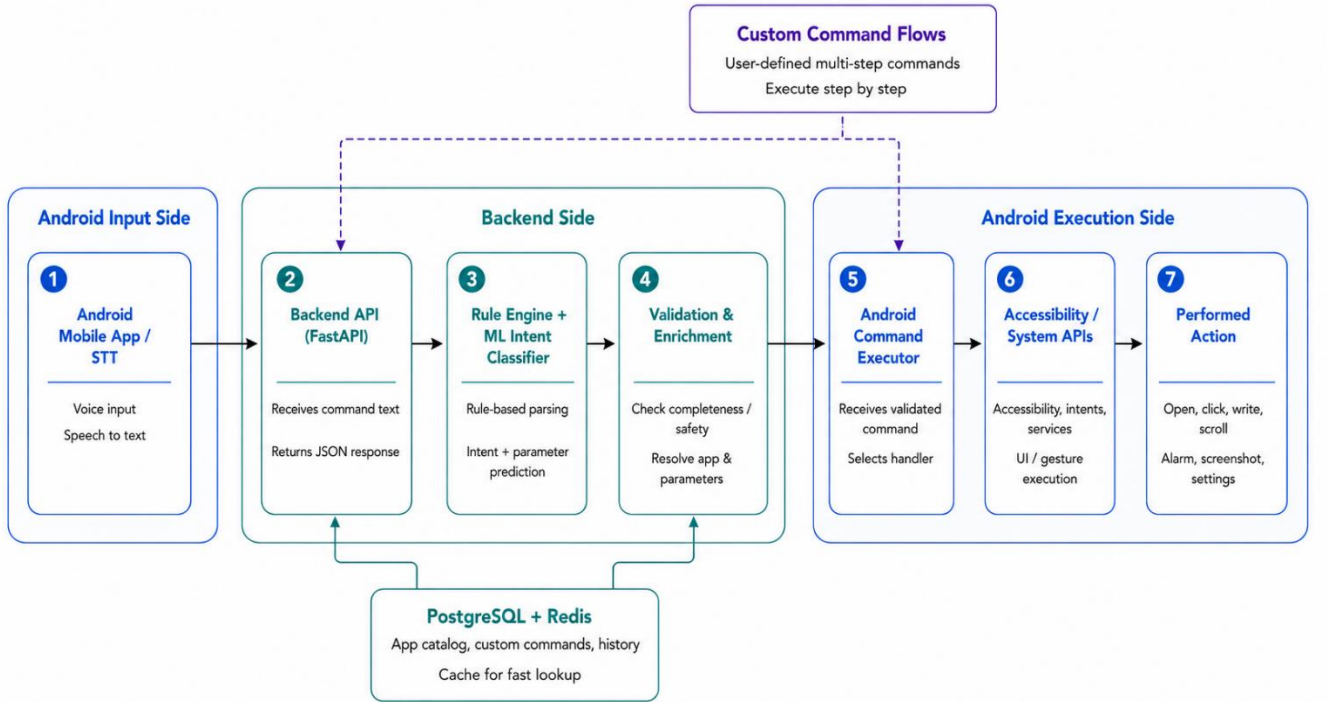
- Android Studio: Uygulama geliştirme ortamı
- Android SDK: Uygulama geliştirme araçları
- Android Accessibility Service API: Kullanıcı arayüzü ile etkileşimler
- Doğal Dil İşleme (NLP): Metin analizi ve ön işleme
- Speech-To-Text (STT): Komutların metne çevrilmesi
- Makine Öğrenmesi Kütüphaneleri: Model geliştirme
- Python (FastAPI): Model için backend
- PostgreSQL: Veri depolama için veritabanı
- Redis: Veritabanı yükünü hafifletmek (cache)

4.1.3. ANDROID UYUMLULUĞU VE SİSTEM GEREKSİNİMLERİ

Android uygulaması minimum SDK 24 ve hedef SDK 35 değerleriyle yapılandırılmıştır. Bu nedenle uygulamanın Android 7.0 ve üzerindeki cihazlarda çalışması beklenmektedir. Sistemin kullanılabilirliği için mikrofon, internet bağlantısı ve gerekli Android erişilebilirlik izinlerini destekleyen bir cihaz gerekmektedir. Bununla birlikte performans ve bazı sistem işlemlerinin kullanılabilirliği cihaz üreticisine, Android sürümüne ve donanım özelliklerine göre farklılık gösterebilir.

4.2. YÖNTEM

Bu projede, sesli komutların Android cihaz üzerinde uygulanabilir bir işleme dönüştürülmesi için modüler ve katmanlı bir sistem mimarisi kullanılmıştır. Sistem genel olarak Android uygulaması, backend API, makine öğrenmesi modeli, kural tabanlı çözümleme, doğrulama katmanı, veritabanı/cache yapısı ve Android komut yürütme katmanlarından oluşmaktadır. Bu katmanlar genel hatlarıyla Şekil 4'te gösterilmiştir.

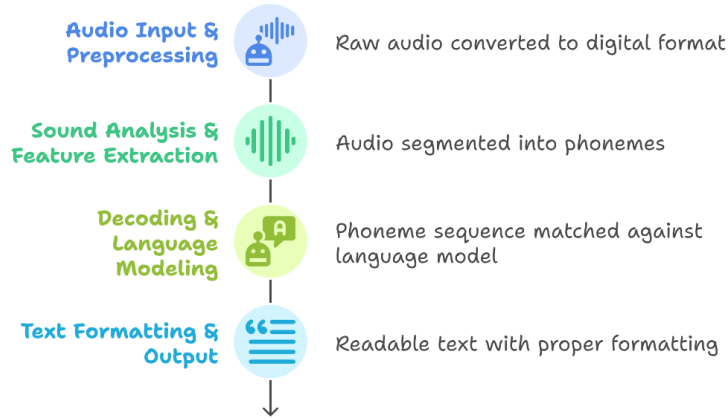


Şekil 4. Sesli komut işleme katmanları

4.2.1. SESLİ KOMUTLARIN METNE DÖNÜŞTÜRÜLMESİ (STT)

Sesli komutlar kullanıcıdan alındıktan sonra Speech-To-Text (STT) modülüyle Android'in sağlamış olduğu SpeechRecognizer API ile metne dönüştürülmektedir (*SpeechRecognizer | API Reference | Android Developers*, n.d.). Bu sürecin temel işleyişi Şekil 5'te gösterilmektedir. Bu süreçte ses sinyalleri işlenerek yazılı bir metin elde edilmektedir.

The Speech-to-Text Process: A Step-by-Step Journey



Şekil 5. STT aşamalarının temel ilerleme aşamaları (*Speech to Text Guide: Convert Audio to Text 3x Faster (2025)*, n.d.)

Alınan komutların STT ile doğru bir şekilde metne dönüştürülmesi kritik bir rol oynar. STT süreci, İngilizce, Türkçe ve Arapça sesli komutların metne dönüştürülmesine olanak sağlamaktadır.

Komutlar başarılı bir şekilde metne dönüştürüldükten sonra metnin analiz edilmesi için backend'e aktarılır.

4.2.2. METİN ÖN İŞLEME, NORMALİZASYON VE KURAL TABANLI ÇÖZÜMLEME (BACKEND)

STT aşamasında elde edilen metin, backend tarafında çeşitli ön işleme adımlarından geçirilmektedir. Bu adımların temel amacı, kullanıcı komutlarının STT tarafından farklı biçimlerle, büyük küçük harf, noktalama işaretleri ve dil bazlı karakter değişikliklerinden kaynaklanan tutarsızlıkları azaltmaktır.

Ön işleme adımlarında dil bazlı karakter normalizasyonları ve karakter temizliği de

uygulanmıştır. Özellikle Arapça komutlarda harf varyasyonları ve Türkçe dilinde komutlara eklenen eklerden kaynaklanan farklılıklar göz önünde bulundurularak komutların daha esnek ve doğru bir şekilde işlenmesi sağlanmıştır. Bu yaklaşım açık uçlu parametrelerin model tarafından ezberlenmesi yerine backend tarafında dinamik olarak çıkarılmasını sağlamaktadır.

Uygulama adı eşleştirme sürecinde yalnızca doğrudan metin karşılaştırması yapılmamış; normalleştirilmiş metin varyantları, Arapça fonetik benzerlikler ve heceleme/kodlama biçiminde söylenen uygulama adları da dikkate alınmıştır. Bu sayede STT çıktısındaki küçük yazım farklılıklarına rağmen uygulama eşleştirme sürecinin daha esnek çalışması sağlanmıştır.

Bu aşamada tek başına makine öğrenmesi modeline başvurulmamaktadır. Belirgin olup değişiklik göstermeyen ve sıkça kullanılan komutlar için kural tabanlı çözümleme mekanizması geliştirilmiştir. Örnek olarak kaydırma yapma, uygulama açma ve zamanlayıcı kurma gibi komutlarda parametre çıkarımı kural tabanlı yöntemlerle sağlanmaktadır.

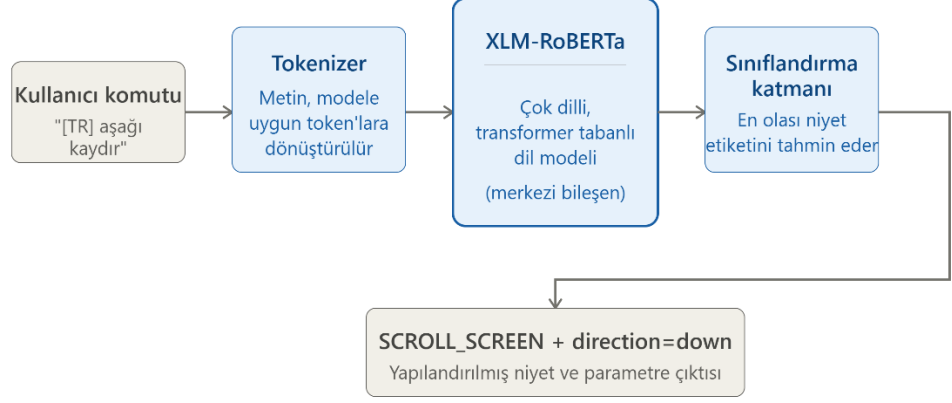
4.2.3. KOMUT SINIFLANDIRMA (MAKİNE ÖĞRENMESİ)

Kural tabanlı yöntemlerle kesin olarak çözülmeyen veya doğal dil yapısı değişkenlik gösteren komutlar için makine öğrenmesi tabanlı intent sınıflandırma modeli geliştirilmiştir (*XLM-RoBERTa* · *Hugging Face*, n.d.). Modelin çoklu dili destekleyebilmesi için transformers XLM-RoBERTa modeli tercih edilmiştir (Conneau et al., 2020). Modelin çalışma akışı Şekil 6'da gösterilmektedir. Model, İngilizce, Türkçe ve Arapça dillerinden oluşan etiketli veri seti ile eğitilmiştir.

Model eğitiminde FacebookAI/xlm-roberta-base temel modeli kullanılmış ve sınıflandırma katmanı 58 çıktı etiketini tahmin edecek şekilde yapılandırılmıştır. Eğitim 12 epoch boyunca, 8 batch büyüklüğü ve 3×10^{-5} öğrenme oranı kullanılarak gerçekleştirilmiştir.

Tokenize edilen komutların maksimum dizi uzunluğu 96 token olarak

belirlenmiştir. Bu yapılandırmada 33 intent ailesi, kategorik parametrelerle oluşturulan intent-parametre birleşimleri nedeniyle 58 ayrı model etiketiyle temsil edilmiştir.



Şekil 6. XLM-RoBERTa tabanlı niyet sınıflandırma pipeline'ı

Modelin temel amacı, kullanıcıdan gelen komut metnini önceden tanımlanmış olan intent sınıflarından birisiyle eşleştirmektir. Bunun yanı sıra bazı intentlerde sınırlı ve kategorik parametreler de model etiketi içerisinde temsil edilmektedir. Örneğin yön bilgisi, ses modu, kamera tipi veya açma kapatma durumları gibi sınırlı parametrelerin tahmin edilmesi modele bırakılmıştır.

Model eğitimi sırasında metinler tokenizer aracılığıyla modele uygun formata dönüştürülmüş ve sınıflandırma katmanı intent etiketlerini tahmin edilebilecek şekilde ayarlanmıştır. Model performansı accuracy, macro F1, weighted F1 ve intent accuracy metrikleri kullanılarak değerlendirilmiştir. Bu metrikler sayesinde hem genel sınıflandırma hem de az örnekli sınıflar üzerinde performans görülmüştür.

4.2.4. INTENT DOĞRULAMA VE PARAMETRE ÇIKARIMI

Model veya kural tabanlı modüller tarafından elde edilen intent verileri Android tarafına gönderilmeden önce backend'de bulunan doğrulama katmanından geçirilmektedir. Bu katmanda her intent için beklenen parametre, zorunlu alanlar, opsiyonel değerler ve Android tarafında desteklenmiş olup olmadığı kontrol edilmektedir.

Bu yapı sayesinde sistem, eksik veya hatalı parametre içeren komutları doğrudan çalıştırmak yerine güvenli bir şekilde kontrol edip reddedebilmektedir. Örnek olarak uygulama açma/silme komutlarında uygulama adı Android mobilden gönderilen uygulama kataloğu ile eşleştirilmekte, alarm komutlarında saat, gün ve gün dilimi bilgisi çıkarılmakta, metin yazma komutlarında yazılacak içerik belirlenmektedir.

Asistanı durdurma gibi yanlış çalışması durumunda kullanıcı deneyimini doğrudan etkileyebilecek intentlerde daha kontrollü bir yaklaşım benimsenmiştir. Bu tür komutlarda model çıktısına tamamen bağlı kalmak yerine kural tabanlı eşleşmeler ve doğrulama kontrolleri kullanılarak hatalı tetiklenmeler azaltılmaya çalışılmıştır.

Doğrulama katmanı ayrıca desteklenmeyen, eksik parametrelili veya Android tarafında güvenli şekilde çalıştırılmayan komutların ayıklanmasını sağlamaktadır. Böylece sistemde intent tahmini, parametre çıkarımı ve komut işleme sorumlulukları birbirinden ayrılmış olmaktadır. Bu yaklaşım, sistemin genişletilebilir olmasını ve yeni komutların kontrol altına alınmasını sağlamaktadır.

4.2.5. MODEL SERVİSLEME VE BACKEND API (FASTAPI)

Eğitilen makine öğrenmesi modelinin Android mobil uygulama tarafından kullanılabilmesi için Python tabanlı FastAPI framework'ü ile bir backend API geliştirilmiştir (*FastAPI - FastAPI*, n.d.). Bu API, Android uygulamasından gelen komutları alıp gereken işlemleri gerçekleştirip sonucu JSON formatında Android tarafına göndermektedir.

Backend üzerinde temel komut tahmini için /predict endpoint'i kullanılmaktadır. Android uygulaması bu endpoint'e kullanıcının komut metnini, dil bilgisini ve gerekli durumlarda cihaz veya oturum bilgilerini göndermektedir. Backend cevabında intent, parametre, güven değeri, komutun kabul edilip edilmediği, eksik parametreler ve hata kodu gibi bilgiler yer almaktadır.

Bunun yanında uygulama kataloğunun backend'e gönderilip uygulama eşleştirilmesi ve kullanıcının belirleyebileceği özel komutlar için ayrı endpoint'ler kullanılmaktadır. Şekil 7'de API'da kullanılan endpoint'ler gösterilmektedir.

Android Assistant Intent API 0.1.0 OAS 3.1
[/openapi.json](#)

default		^
GET	/ Root	▼
POST	/predict Predict	▼
GET	/custom-commands Custom Commands	▼
POST	/custom-commands Create Custom Command	▼
PUT	/custom-commands/{command_id} Edit Custom Command	▼
DELETE	/custom-commands/{command_id} Remove Custom Command	▼
POST	/app-catalog App Catalog	▼
GET	/app-catalog/{session_id} App Catalog Status	▼
DELETE	/app-catalog/{session_id} Close App Catalog	▼

Şekil 7. Backend FastAPI endpoint'leri

GET / : Backend servisinin çalışır durumda olup olmadığını kontrol eder.

POST /predict : Android uygulamasından gelen komut metnini işleyerek intent, parametreler ve kabul durumunu döndürür.

GET /custom-commands : Cihaza ait kayıtlı özel komutları listeler.

POST /custom-commands : Kullanıcı tarafından oluşturulan yeni özel komut akışını veritabanına kaydeder.

PUT /custom-commands/{command_id} : Mevcut bir özel komutun adını, dilini veya adımlarını günceller.

DELETE /custom-commands/{command_id} : Kayıtlı bir özel komutu veritabanından siler.

POST /app-catalog : Android cihazda yüklü uygulama listesini backend'e göndererek uygulama adı ve package name eşleştirmesi için kaydeder.

GET /app-catalog/{session_id} : Belirli bir oturum veya cihaz için uygulama kataloğunun backend tarafında mevcut olup olmadığını kontrol eder.

DELETE /app-catalog/{session_id} : Belirtilen oturum veya cihaz için Redis üzerinde tutulan geçici uygulama kataloğu önbelleğini temizler. Bu endpoint PostgreSQL'deki cihaz ve uygulama kataloğu kayıtlarını silmez. Üç gün boyunca aktif olmayan cihazlara ait kalıcı kayıtlar, backend başlangıcında ve katalog eşitleme işlemleri sırasında çalışan ayrı bir temizleme mekanizması tarafından kaldırılmaktadır.

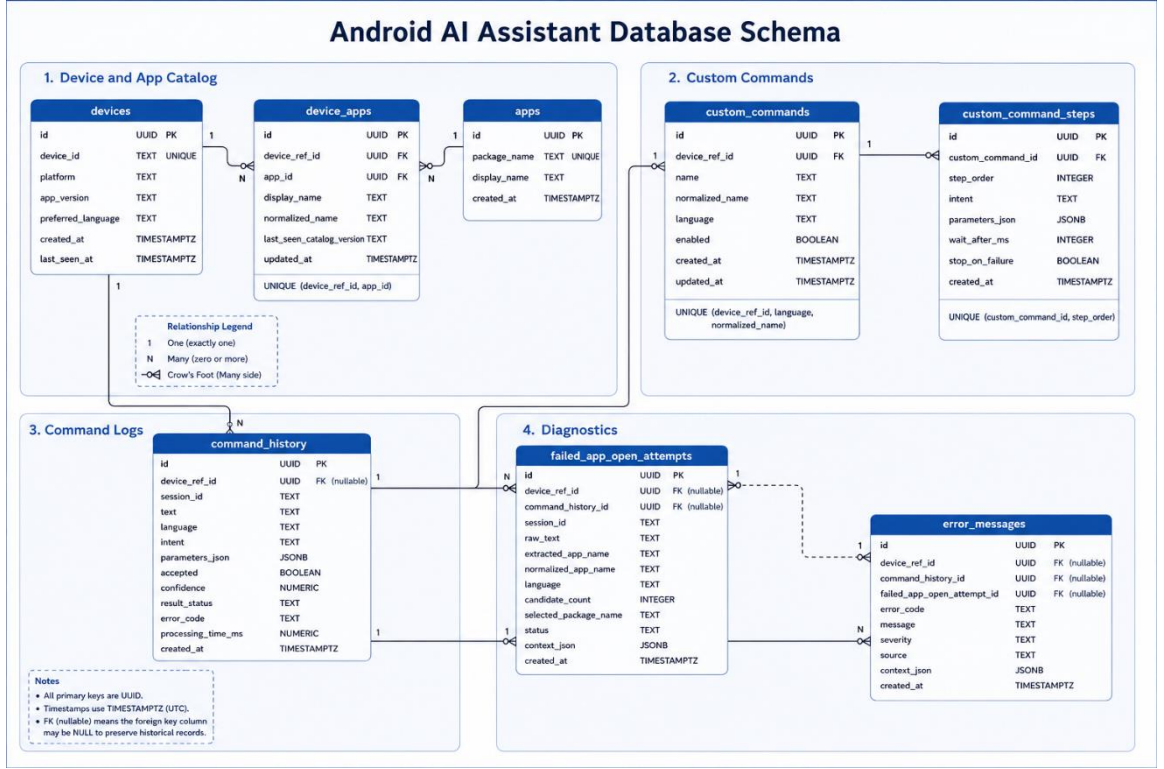
4.2.6. VERİTABANI VE CACHE

Sistemde yalnızca komut gerçekleştirme yapılmamakta, bazı verilerin kalıcı veya geçici olarak saklanması da gerekmektedir. Bu amaçla PostgreSQL veritabanı ve Redis cache yapısı kullanılmıştır (*PostgreSQL: Documentation: 18: 1. What Is PostgreSQL?*, n.d.). PostgreSQL tarafında cihaz bilgileri, uygulama kataloğu, özel komutlar ve komut geçmişi gibi veriler tutulmaktadır.

Android cihazda yüklü olan uygulamaların listesi backend'e gönderilmekte olup, uygulama açma, uygulama bilgisi görüntüleme veya uygulama silme gibi işlemlerde gönderilen katalogdan yararlanılmaktadır. Böylece kullanıcının söylediği uygulama adı, cihazda yüklü olan gerçek uygulama paketleriyle eşleştirilebilmektedir.

Redis ise sık erişilen uygulama kataloğu verilerinin daha hızlı kullanılabilmesi için cache katmanı olarak kullanılmaktadır (*Quick Starts | Docs*, n.d.). Redis kalıcı veri kaynağı olarak değil, veritabanı üzerindeki yükü azaltan geçici bir performans katmanı olarak yer almaktadır. Veritabanı tabloları, ait oldukları kategori ve aralarındaki ilişki Şekil 8'de olduğu gibi gösterilmektedir.

Komut geçmişi `command_history` tablosunda saklanmaktadır. `failed_app_open_attempts` ve `error_messages` tabloları ise gelecekteki hata analizi süreçleri için şema düzeyinde hazırlanmıştır. Ayrıca belirli süre aktif olmayan cihaz kayıtları temizlenerek uygulama kataloğu verilerinin gereksiz şekilde birikmesi engellenmektedir.



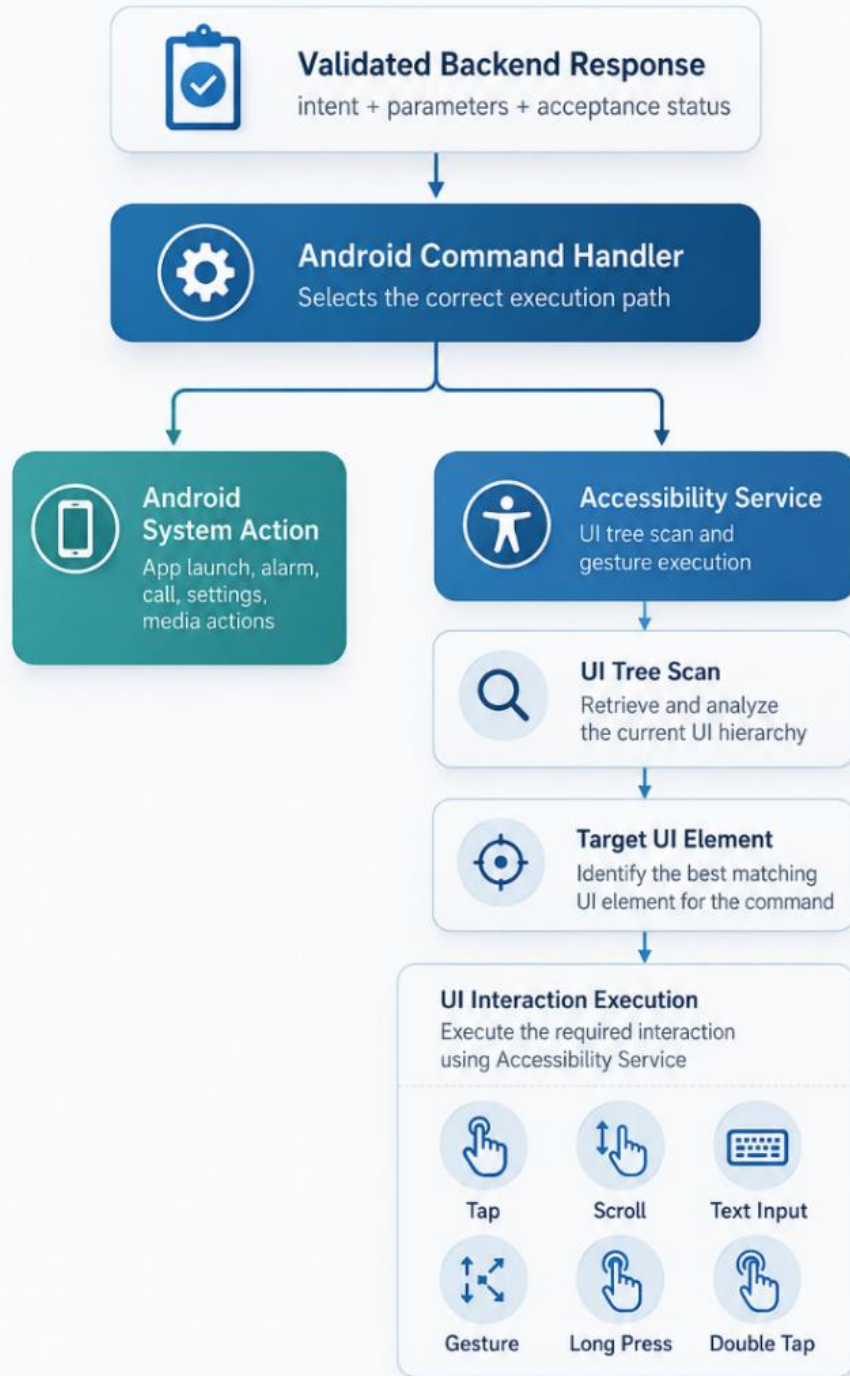
Şekil 8. Veritabanı ERD Diagramı

Bu veritabanı yapısında devices tablosu Android cihazları, apps tablosu genel uygulama sözlüğünü, device_apps tablosu ise cihazlarda yüklü uygulama kataloglarını temsil etmektedir. Özel komutlar custom_commands ve custom_command_steps tablolarında adım sırası korunarak saklanmaktadır. command_history tablosu ise işlenen komutların sonuçlarını kaydetmek için kullanılmaktadır.

4.2.7. ANDROID ERİŞİLEBİLİRLİK SERVİSİ İLE ARAYÜZ ETKİLEŞİMİ

Backend tarafında doğrulanan ve kabul edilen komutlar Android uygulamasına yapılandırılmış JSON formatında gönderilmektedir. Android tarafında bu komutlar ilgili command handler'ı tarafından karşılanmakta ve komutun türüne göre farklı yürütme mekanizmaları kullanılmaktadır. Şekil 9'da görüldüğü gibi bu akışın en genel hatlarıyla bu şekilde gerçekleştiği düşünülebilir.

Android Command Execution Flow

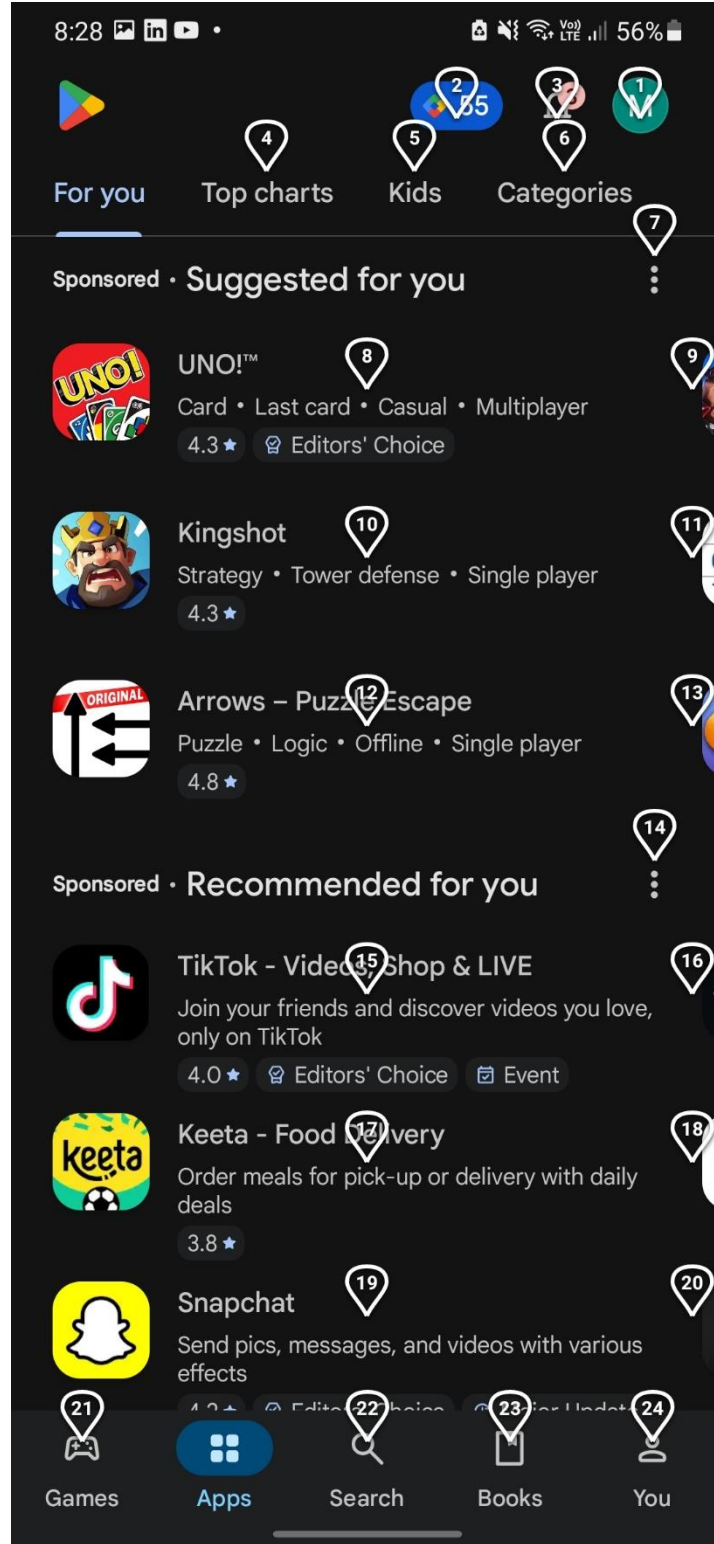


Şekil 9. Android Komut yürütme akışı

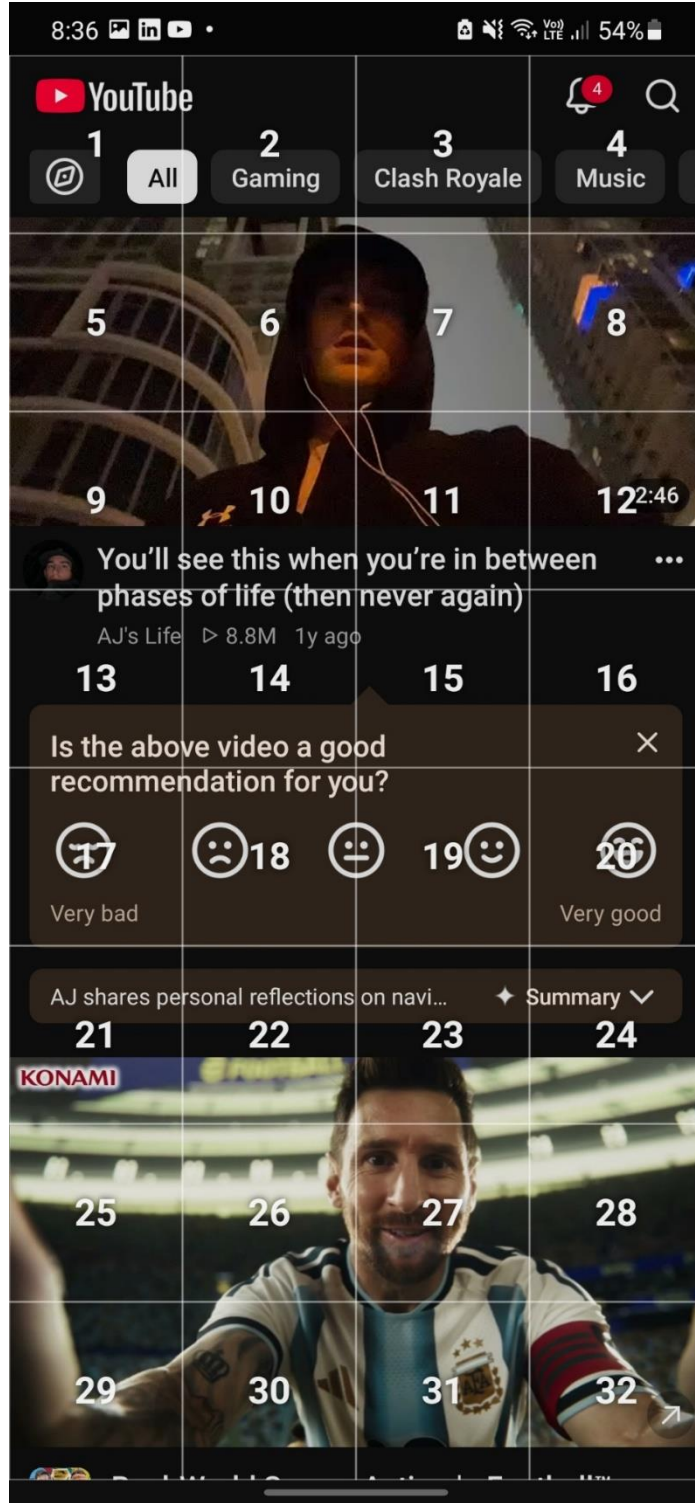
Uygulama açma, alarm kurma, arama başlatma veya sistem ayarlarına yönlendirme gibi bazı işlemler Android'in sunduğu intent ve sistem servisleriyle gerçekleştirilmektedir. Kaydırma, tıklama, metin alanına yazma, ekran üzerinde eleman arama ve benzeri arayüz etkileşimleri ise Android Erişilebilirlik Servisi aracılığıyla yapılmaktadır. Erişilebilirlik Servisi sayesinde aktif uygulamanın kullanıcı arayüzü ağacı incelenebilmekte ve uygun kullanıcı arayüzü bileşenlerine dokunma, uzun basma, çift tıklama veya metin yazma gibi işlemler uygulanabilmektedir.

Bazı sistem ayarları Android API'leri üzerinden doğrudan değiştirilemediği için alternatif bir yürütme akışı kullanılmıştır. Bu durumda hızlı ayarlar paneli açılmakta, ilgili Wi-Fi, Bluetooth, konum, mobil veri veya hotspot bileşeni erişilebilirlik ağacı üzerinden bulunmakta ve uygun durum değişikliği kullanıcı arayüzü üzerinden gerçekleştirilmektedir.

Dinamik kullanıcı arayüzlerinde hedef elemanın doğrudan bulunamadığı durumlarda yardımcı yöntemler de kullanılmaktadır. Örneğin ekrandaki erişilebilir öğelerin numaralandırılması (Şekil 10), grid yapısı (Şekil 11) ile ekran bölgelerinin seçilmesi veya aday seçeneklerin kullanıcıya sunulması gibi yöntemler, farklı uygulamalarda komutların daha esnek çalışmasına katkı sağlamaktadır. Bununla birlikte, üçüncü taraf uygulamaların erişilebilirlik bilgilerini farklı şekillerde sunması nedeniyle bazı arayüz etkileşimlerinde cihaz, uygulama ve ekran durumuna bağlı farklılıklar oluşabilmektedir.



Şekil 10. Android erişilebilirlik (Numaralandırma örneği)



Şekil 11. Android erişilebilirlik (Grid örneği)

4.3. GELİŞMİŞ KOMUT ETKİLEŞİMLERİ

4.3.1. DİNAMİK KULLANICI ARAYÜZÜ ETKİLEŞİMİ

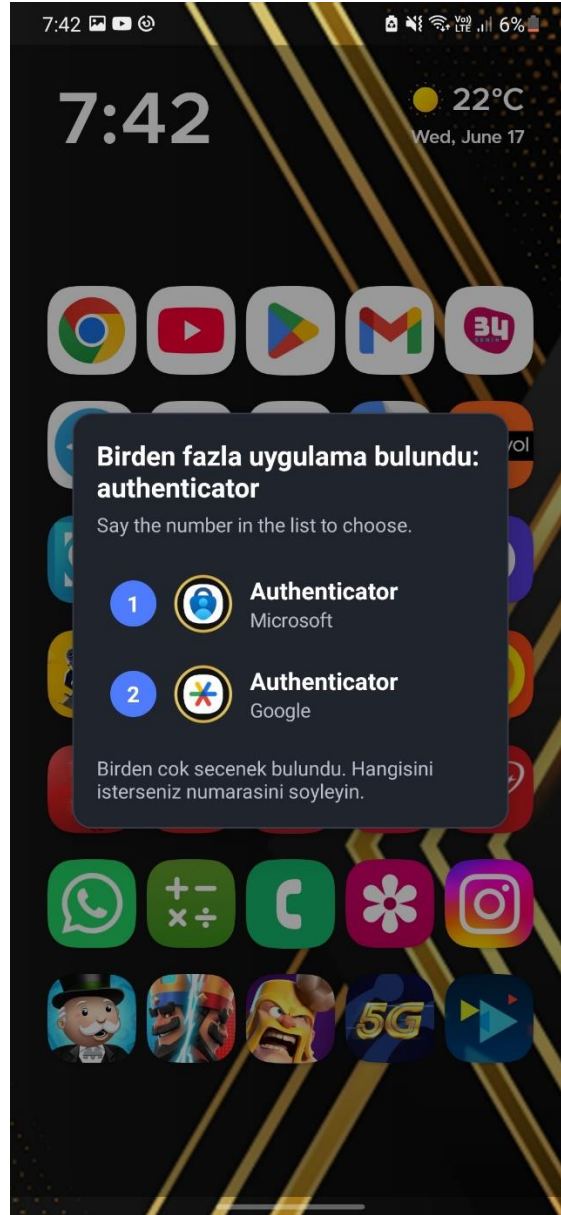
Bu çalışmada komutların yalnızca sabit sistem işlemlerine dönüştürülmesi değil, açık olan uygulama ekranındaki dinamik arayüz elemanlarıyla da etkileşim kurulması sağlanmıştır. Android erişilebilirlik servisi ile ekrandaki metinler, butonlar, ikonlar ve input alanları taranarak kullanıcının komutunda belirttiği hedef eleman bulunmaya çalışılır. Böylece sistem sabit koordinatlara bağlı kalmadan, ekrandaki gerçek UI bileşenleri üzerinde işlem yapılabilir.

Eğer hedef bileşen tek bir sonuçla eşleşirse işlem doğrudan gerçekleştirilir. Ancak aynı UI bileşeninden birden fazla bulunursa adaylar ekran üzerinde numaralandırılır ve kullanıcıdan seçim yapması istenir. Örneğin Şekil 12’de görüldüğü gibi Chrome uygulamasında hedef akışı içerisinde kullanıcı paylaş butonuna basmak istediğinde, ekranda birden fazla paylaş simgesi bulunabilmektedir. Bu durumda sistem tüm paylaş adaylarını ekran üzerinde numaralandırarak gösterir ve kullanıcı seçmek istediği paylaş butonunu numarasıyla belirler. Böylece aynı türden birden fazla arayüz öğesi bulunan ekranlarda yanlış bileşene tıklanmasının önüne geçilir.



Şekil 12. Chrome'da aday numaralandırma örneği

Bu yapı aynı şekilde uygulama açma, input alanı seçme ve başka birçok akışta aynı mantıkta kullanılmaktadır. Bu doğrultuda uygulama açma ve input alanı seçme işlemlerine Şekil 13 ve Şekil 14’te olduğu gibi örnek verilebilir. Şekil 13’te cihazda iki Authentication uygulaması kurulu olmasından dolayı bir pencere içerisinde kullanıcıya bu iki uygulama arasında seçme hakkı tanınır. Şekil 14’te ise bir metin yazmak istendiği zaman metnin hangi input alanına yazılacağı kullanıcıya seçtirilir. Kullanıcının seçimine göre metin ilgili input alanına yazılır.



Şekil 13. Uygulama açma akışında aday seçtirme örneği

Şekil 14. Yazı girme akışında aday input seçtirme örneği

4.3.2. ÖZEL KOMUT AKIŞLARI

Özel komut akışları, kullanıcının birden fazla komutu tek bir özel komut adı altında sıralı şekilde çalıştırabilmesini sağlar. Kullanıcı uygulama içinde bir akış oluşturur, bu akışa bir ad verilir ve daha sonra bu adı sesli olarak söylediğinde sistem adımları sırayla gerçekleştirir. Bu yapı özellikle tekrar eden işlemleri daha hızlı gerçekleştirmek için kullanılabilmektedir.

Her özel komut birden fazla adımdan oluşabilir. Bu adımlar uygulama açma, arama yapma, ekrandaki bir öğeye tıklama, bekleme süresi ekleme veya numaralandırılmış bir öğeyi seçme gibi işlemleri içerebilir. Özel komutlar backend tarafında PostgreSQL veritabanında saklanır ve Android tarafında sırayla yürütülür. Eğer bir adımda hata oluşursa kalan adımlar çalıştırılmaz ve akış durdurulur. Örnek olarak kullanıcı “özel komut: salla kazan yap” derse kullanıcının daha önceden belirlemiş olduğu özel komut akışı gerçekleştirilmeye başlar. Bu akış, aşağıdaki gibidir:

1.adım: Uygulama açma → Turkcell

2.adım: Uygulama açılana kadar bekleme → 15 saniye

3.adım: UI üzerinden “salla kazan” sekmesi seçilir → 15 numaralı bileşen

4.adım: Sekme açılana kadar bekleme → 1 saniye

5.adım: UI üzerinden “Şimdi salla kazan” butonuna basma → 2 numaralı bileşen

Bu aşamalar sonrasında artık kullanıcı telefonu sallayarak işlemini tamamlayabilir.

5.SONUÇLAR

Proje sonucunda Android uygulaması, FastAPI backend servisi ve XLM-RoBERTa tabanlı komut sınıflandırma modeli birlikte çalışan uçtan uca bir sistem haline getirilmiştir. Android uygulaması kullanıcı komutlarını SpeechRecognizer ile metne dönüştürmekte, backend'e göndermekte ve kabul edilen intent çıktısını AccessibilityService, Android sistem servisleri ve komut handler yapısı üzerinde çalıştırmaktadır. Backend tarafında önce kural çözümleme, ardından makine öğrenmesi modeli ve son olarak intent doğrulama ve parametre zenginleştirme katmanı çalışmaktadır.

5.1. GEREKSİNİM LİSTESİ

Bu proje için gereksinim listesi fonksiyonel ve fonksiyonel olmayan gereksinimler olarak iki başlık altında incelenmiştir.

5.1.1. FONKSİYONEL GEREKSİNİMLER

Fonksiyonel gereksinimler, sistem hangi görevleri yerine getirmelidir? Sorusunun cevabıdır. Bu gereksinimler Tablo 2'de listelenmektedir.

Tablo 2. Fonksiyonel gereksinimler listesi

FG1	Kullanıcıdan alınan komutlar STT aracılığıyla metne dönüştürülmelidir
FG2	Komut metni üzerinde dil bazlı normalizasyon, kural tabanlı çözümleme, parametre çıkarımı ve doğrulama işlemleri yapılmalıdır
FG3	Kural tabanlı çözülmeyen komutlar XLM-RoBERTa tabanlı makine öğrenmesi modeliyle sınıflandırılmalıdır
FG4	Farklı yazıma sahip ancak aynı anlama gelen komutların aynı intent sınıfı altında değerlendirilmelidir
FG5	Intent bilgisi Android Erişilebilirlik Servisi aracılığıyla kullanıcı arayüzü etkileşimlerine dönüştürülmelidir

FG6	Uygulamadan bağımsız olarak kullanıcı arayüzü bileşenleriyle etkileşim sağlanmalıdır
FG7	Uygulama açma/silme, arayüz öğesine tıklama, kaydırma, metin yazma, input alanına odaklanma/odaktan çıkarma, arama yapma, alarm/zamanlayıcı kurma, fotoğraf, ekran görüntüsü, aramayı cevaplama/reddetme, medya oynatmayı devam ettirme, telefonu kapatma/yeniden başlatma ve sistem ayarları gibi görevler yerine getirilmelidir
FG8	Çoklu dil desteklenmelidir: İngilizce, Türkçe ve Arapça
FG9	Uygulama ile ilgili işlemlerin gerçekleştirilebilmesi için cihazdan gelen uygulama kataloğu kullanılmalıdır
FG10	Birden fazla aday bulunan dinamik arayüz elemanları kullanıcıya seçtirilebilmelidir
FG11	Kullanıcı, birden fazla adımdan oluşan özel komut akışları oluşturabilmeli, düzenleyebilmeli, silebilmeli ve çalıştırabilmelidir
FG12	Sistem; cihaz, uygulama kataloğu, özel komut ve komut geçmişi verilerini veritabanında saklayabilmeli, uygulama kataloğu için cache katmanı kullanabilmelidir.

5.1.2. FONKSİYONEL OLMAYAN GEREKSİNİMLER

Fonksiyonel olmayan gereksinimler, sistem nasıl çalışmalıdır? Sorusunun cevabıdır. Bu gereksinimler Tablo 3'te listelenmektedir.

Tablo 3. Fonksiyonel olmayan gereksinimler listesi

FOG1 - Performans	Komut gerçekleştirme süresi kabul edilebilir düzeyde olmalıdır
FOG2 - Doğruluk	Komutların doğru intent ve parametrelerle işlenmesi, hatalı veya eksik komutların güvenli şekilde reddedilmesi sağlanmalıdır

FOG3 - Kaynak Kullanımı	Mobil uygulama tarafında işlem yükü düşük tutulmalı, model tahmini ve veri işleme gibi ağır işlemler backend tarafında yürütülmelidir
FOG4 - Erişilebilirlik	Hareket kısıtı olan bireylerin kullanımına uygun bir şekilde tasarlanmalıdır
FOG5 - Güvenilirlik	Hatalı komut girdilerinde kararlı bir şekilde çalışmaya devam etmelidir
FOG6 - Genişletilebilirlik	Yeni intent, parametre ve komut akışları mevcut mimariyi bozmadan sisteme eklenebilmelidir
FOG7 - Kullanılabilirlik	Kullanıcı, izin yönetimi, özel komutlar, kılavuz ve ayarlar ekranları üzerinden sistemi anlaşılır şekilde yönetebilmelidir
FOG8 - Veri Yönetimi	Sık kullanılan uygulama kataloğu verileri cache katmanı ile desteklenerek veritabanı erişim yükü azaltılmalıdır
FOG9 - Güvenli Yürütme	Eksik parametrelili, düşük güvenli veya desteklenmeyen komutlar Android tarafında çalıştırılmadan önce backend doğrulama katmanı tarafından reddedilmelidir

5.1.3. DESTEKLENEN KOMUT KATEGORİLERİ

Desteklenen komut kategorileri ve bu kategorilere ait örnek intent yapıları Tablo 4'te gösterilmiştir.

Tablo 4. Sistem Tarafından Desteklenen Komut Kategorileri

Kategori	Intent Örnekleri	Açıklama
Uygulama Yönetimi	OPEN_APP, OPEN_APP_INFO, UNINSTALL_APP	Uygulama açma, bilgi ekranı açma ve kaldırma
Navigasyon	GO_BACK, GO_HOME, SHOW_RECENTS, CLOSE_APP	Sistem navigasyonu ve uygulamadan çıkış

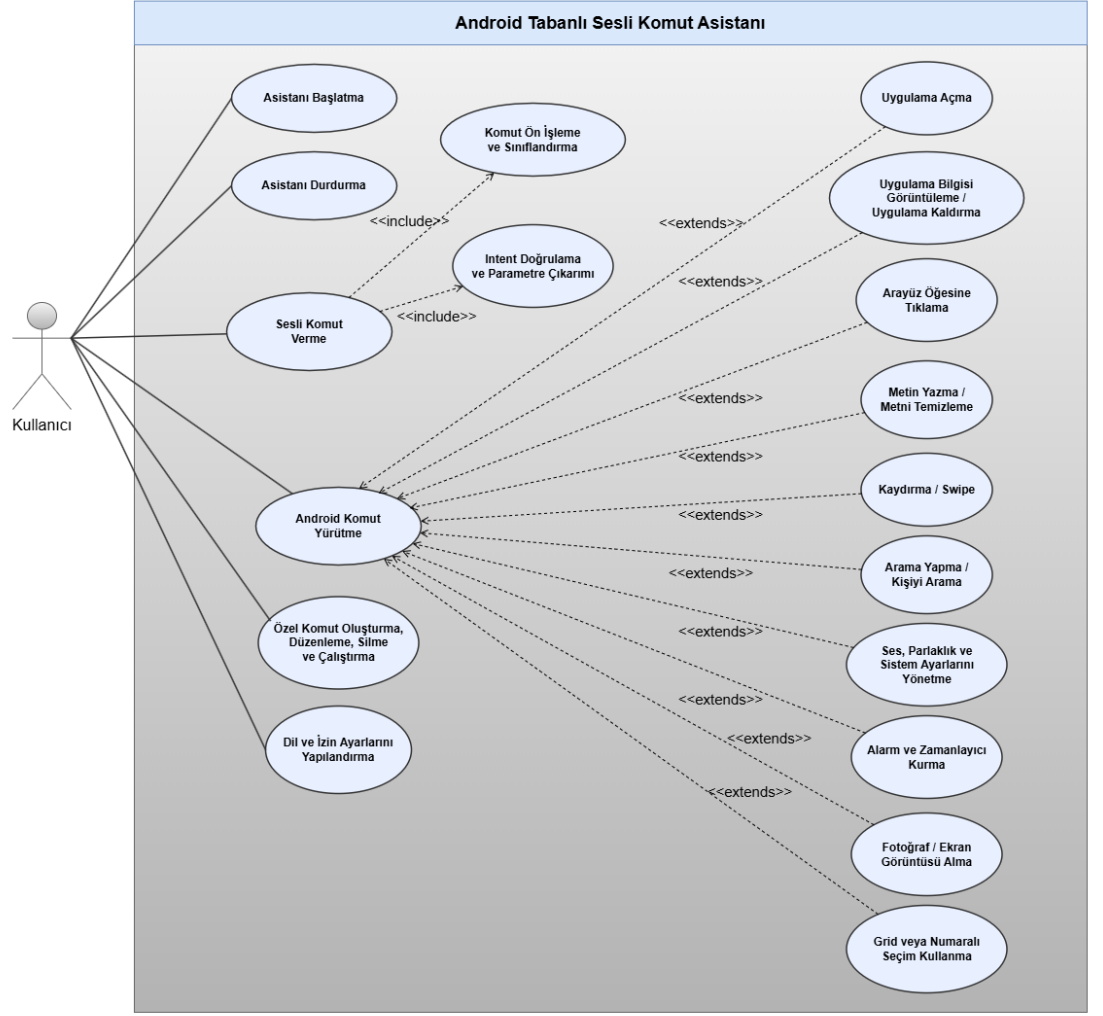
Arayüz Etkileşimi	CLICK_ITEM, WRITE_TEXT, CLEAR_TEXT, SET_INPUT_FOCUS	UI elemanı seçme ve metin alanı işlemleri
Ekran Hareketleri	SCROLL_SCREEN, SWIPE_GESTURE, DOUBLE_TAP, HOLD_SCREEN	Kaydırma, sürükleme ve dokunma hareketleri
Sistem Ayarları	SET_WIFI, SET_BLUETOOTH, SET_LOCATION, SET_SOUND_MODE	Android ayarlarının kontrolü
Medya ve Cihaz	SET_MEDIA_PLAYBACK, TAKE_SCREENSHOT, POWER_OFF, RESTART_DEVICE	Medya, ekran görüntüsü ve güç işlemleri
Zaman ve Arama	SET_ALARM, SET_TIMER, SEARCH_QUERY, CALL_CONTACT	Alarm, zamanlayıcı, arama ve kişi arama
Gelişmiş Akışlar	SHOW_GRID, SHOW_LABELS, RUN_CUSTOM_COMMAND	Grid, numaralandırma ve özel komut akışları

5.2. SİSTEM AKIŞ DİAGRAMLARI

Bu bölümde, sistemin kullanım senaryolarını, modüller arası ilişkiler, dağıtım mimarisini ve işlem sırasını açıklayan diagramlar verilmiştir.

5.2.1. USE CASE DİAGRAM

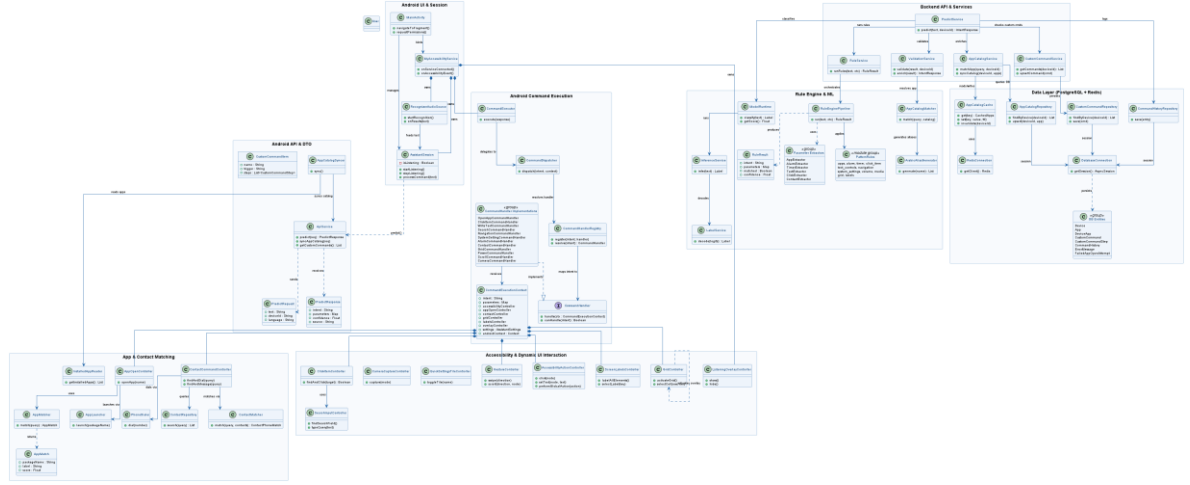
Kullanıcının sistemle etkileşime geçtiği temel senaryolar Şekil 15’te olduğu gibi gösterilmektedir. Bu senaryolar arasında sesli komut verme, uygulama açma, arayüz elemanına tıklama, metin yazma, sistem ayarlarını değiştirme, özel komut oluşturma ve özel komut çalıştırma gibi işlemler yer almaktadır.



Şekil 15. Sistemin genel use case diagramı

5.2.2. CLASS DIAGRAM

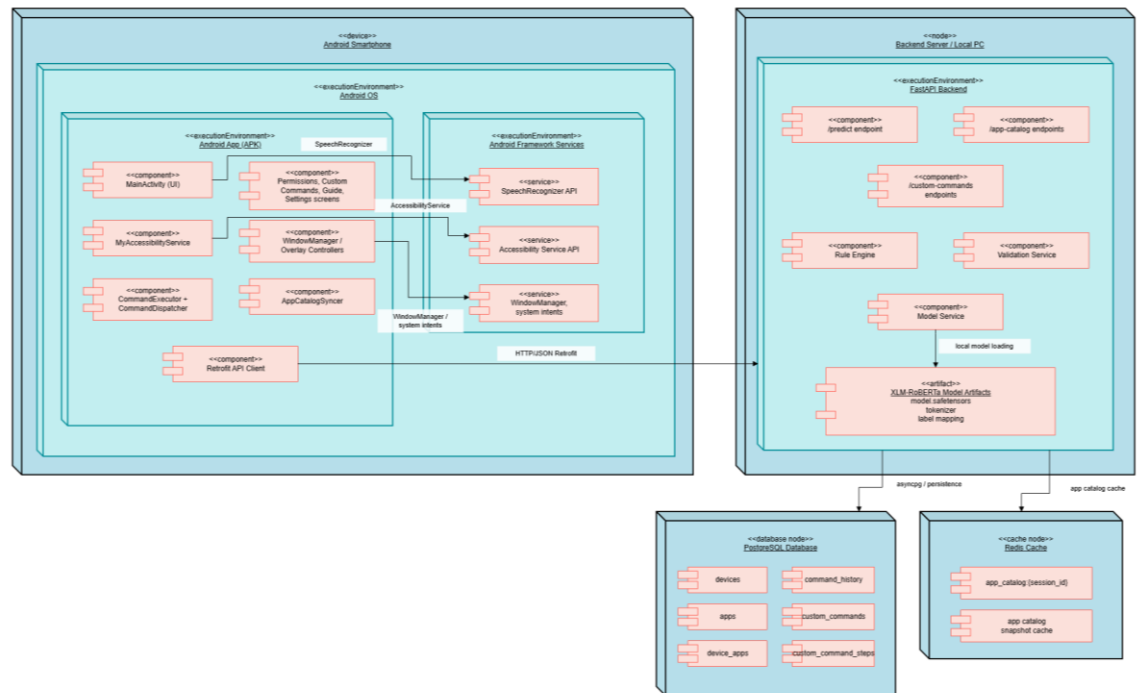
Sistem mimarisindeki temel Android, backend, komut işleme, doğrulama ve veri yönetimi bileşenleri arasındaki ilişkiler Şekil 16'da gösterilmektedir.



Şekil 16. Sistem mimarisini içeren class diagramı

5.2.3. DEPLOYMENT DIAGRAM

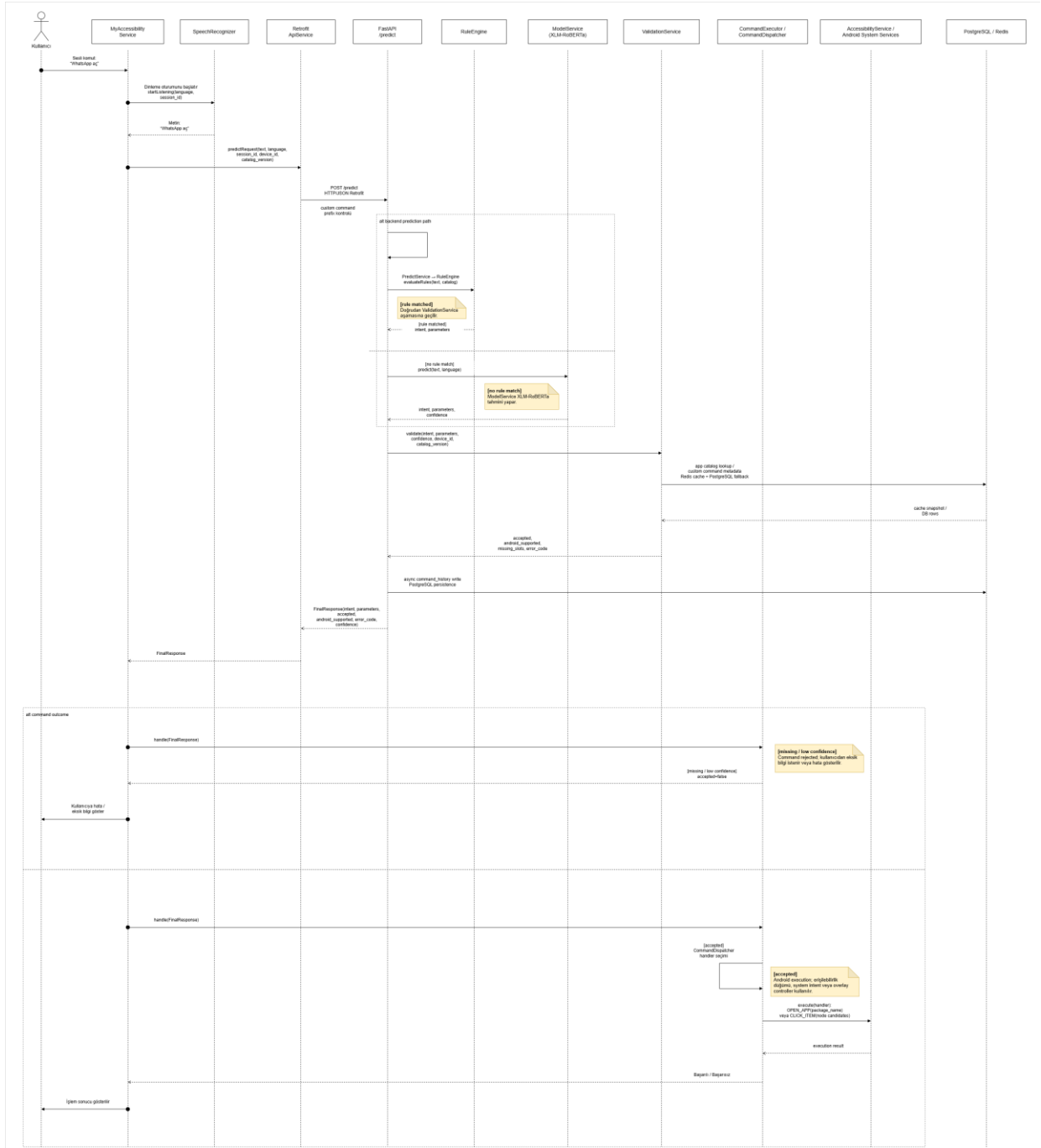
Şekil 17’de Android uygulaması, FastAPI backend servisi, makine öğrenmesi modeli, PostgreSQL veritabanı ve Redis cache katmanının hangi ortamda çalıştığı ve birbirleriyle nasıl iletişim kurduğu gösterilmektedir.



Şekil 17. Sistem bileşenlerinin deployment diagramı

5.2.4. SEQUENCE DIAGRAM

Şekil 18’de kullanıcının sesli komut vermesiyle başlayan süreç gösterilmektedir. Bu süreçte komut Android tarafında metne dönüştürülmekte, backend API’ye gönderilmekte, rule engine veya makine öğrenmesi modeli tarafından işlenmekte, doğrulama katmanından geçirilmekte ve ardından Android tarafında ilgili komut yürütücüsü tarafından çalıştırılmaktadır.



Şekil 18. İş akışı ve sistem bileşenleri arasındaki iletişimi ele alan sequence diagramı

5.3. MAKİNE ÖĞRENMESİ MODELİNİN SONUÇ METRİKLERİ

Tablo 5’te model sonuç metrikleri gösterilmektedir. Bu sonuçlar, modelin çok dilli sesli komutları genel olarak başarılı şekilde sınıflandırabildiğini göstermektedir. Macro F1 değerinin yüksek olması, sistemin yalnızca sık görülen sınıflarda değil, daha az örneğe sahip intentlerde de kabul edilebilir bir performans gösterdiğini ortaya koymaktadır.

Tablo 5. Model sonuç metrikleri

Split	Loss	Accuracy	Macro F1	Weighted F1	Intent Accuracy
Val	0.2391	95.87%	95.08%	95.75%	97.49%
Test	0.3743	93.45%	90.93%	93.05%	94.31%

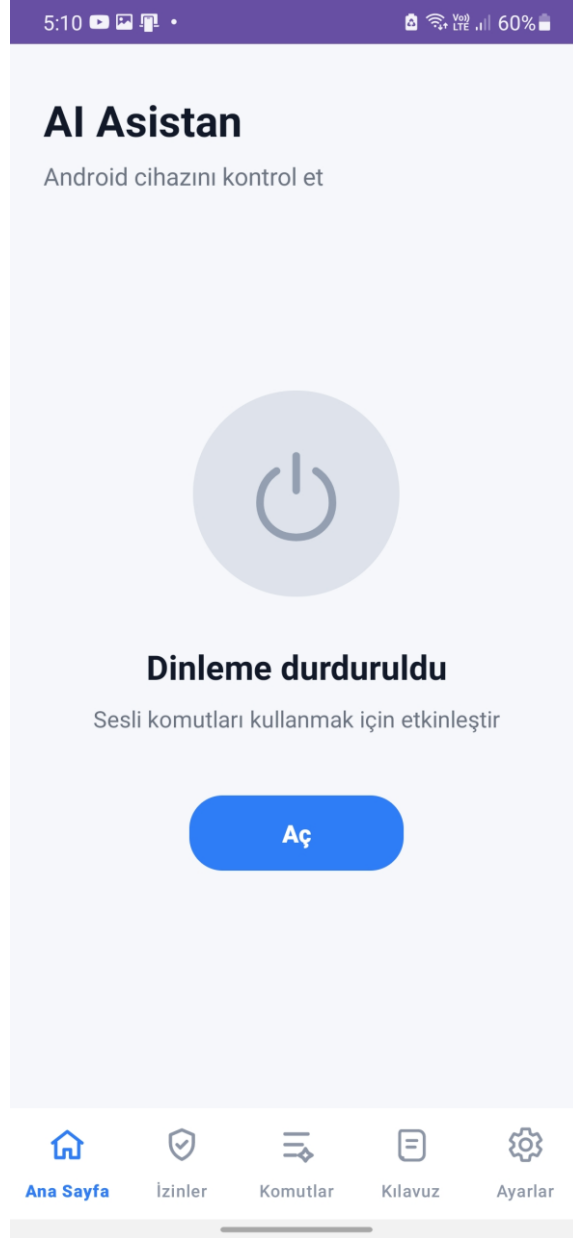
Bununla birlikte, bazı komutların doğal dilde birbirine yakın şekilde ifade edilebilmesi ve bazı intentlerin daha az örnekle temsil edilmesi hata riskini tamamen ortadan kaldırmamaktadır. Bu nedenle sistem yalnızca makine öğrenmesi modeline bağlı bırakılmamış; rule engine, doğrulama katmanı, parametre çıkarımı ve Android tarafındaki güvenli yürütme kontrolleriyle desteklenmiştir.

5.4. MOBİL UYGULAMA KULLANICI ARAYÜZLERİ

Geliştirilen Android uygulaması, asistanın çalışma durumunun yönetilmesi, gerekli izinlerin kontrol edilmesi, özel komut akışlarının oluşturulması, desteklenen sesli komutların incelenmesi ve uygulama ayarlarının yapılandırılması amacıyla beş ana ekrandan oluşmaktadır. Bu ekranlara uygulamanın alt gezinme menüsü üzerinden erişilmektedir. Sesli komutların yürütülmesi arka planda çalışan erişilebilirlik servisi tarafından gerçekleştirilirken mobil arayüz, kullanıcının sistemi yönetmesini ve yapılandırmasını sağlamaktadır.

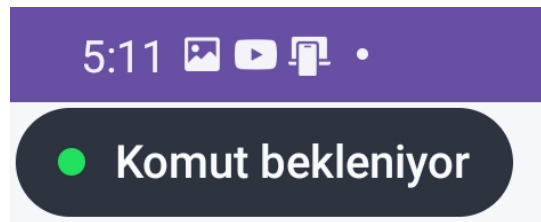
5.4.1. AI ASİSTAN ANA EKRANI

Şekil 19’da gösterilen ekran, uygulamanın başlangıç ekranı ve asistanın temel kontrol noktasıdır. Bu ekranda asistanın aktif veya pasif olduğu bilgisi ve dinleme durumu gösterilmektedir.



Şekil 19. Uygulama içerisinde ana sayfa

Ekranda bulunan düğme ile asistanın başlatılması ve durdurulması sağlanmaktadır. Şekil 20’de ise asistan dinleme durumundayken ekran üzerinde gösterilen arayüz tasarlanmıştır.



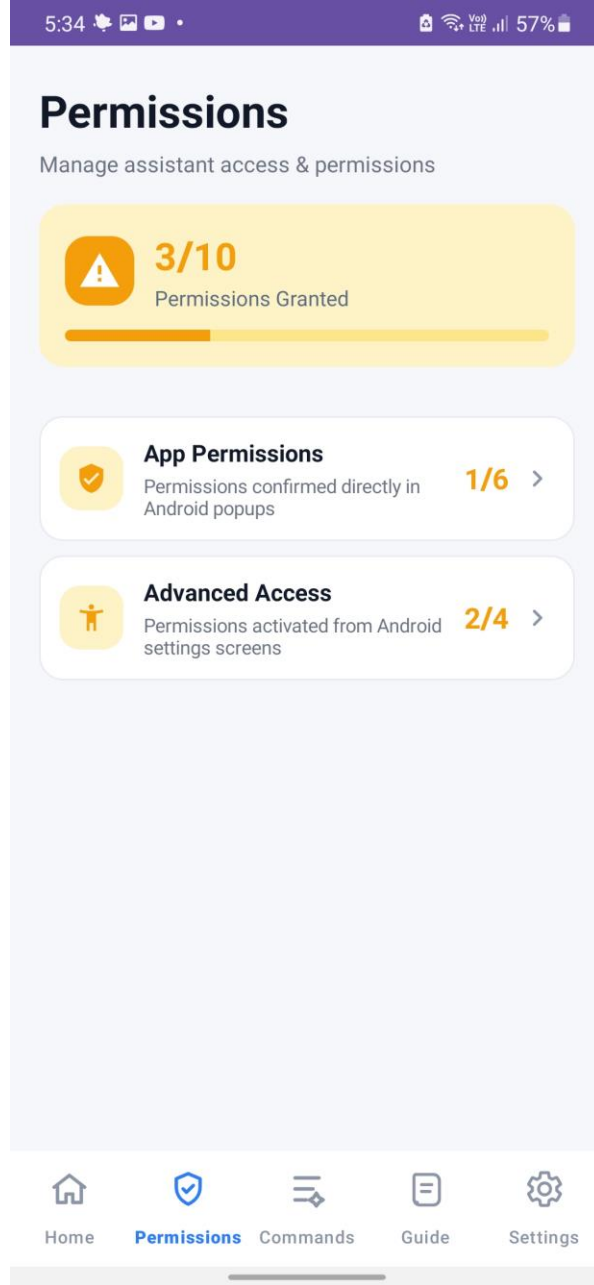
Şekil 20. AI Asistan ekran üstü komut bekleme arayüzü

Asistan başlatılmak istendiğinde uygulama; mikrofon, üstte gösterme ve erişilebilirlik servisi gibi gerekli izinlerin durumunu kontrol etmektedir. Gerekli izinlerden biri eksikse kullanıcı izin yönetimi ekranına yönlendirilmektedir. İzinlerin tamamlanmasının ardından erişilebilirlik servisi üzerinden sesli komut dinleme döngüsü başlatılmakta ve asistanın güncel durumu ana ekran üzerinde kullanıcıya bildirilmektedir. Android SpeechRecognizer oturumları tamamlandığında veya kesintiye uğradığında dinleme işlemi uygulama tarafından yeniden başlatılmaktadır.

5.4.2. İZİN YÖNETİMİ EKRANI

İzin yönetimi ekranı, sesli asistanın işlevlerini gerçekleştirebilmesi için gerekli Android izinlerinin tek bir ekran üzerinden izlenmesini ve yönetilmesini sağlamaktadır. Ekranın üst bölümünde verilen izinlerin genel durumu ve tamamlanma oranı gösterilmektedir. İzinler, “Uygulama İzinleri” ve “Gelişmiş Erişim” olmak üzere iki kategori altında sunulmaktadır.

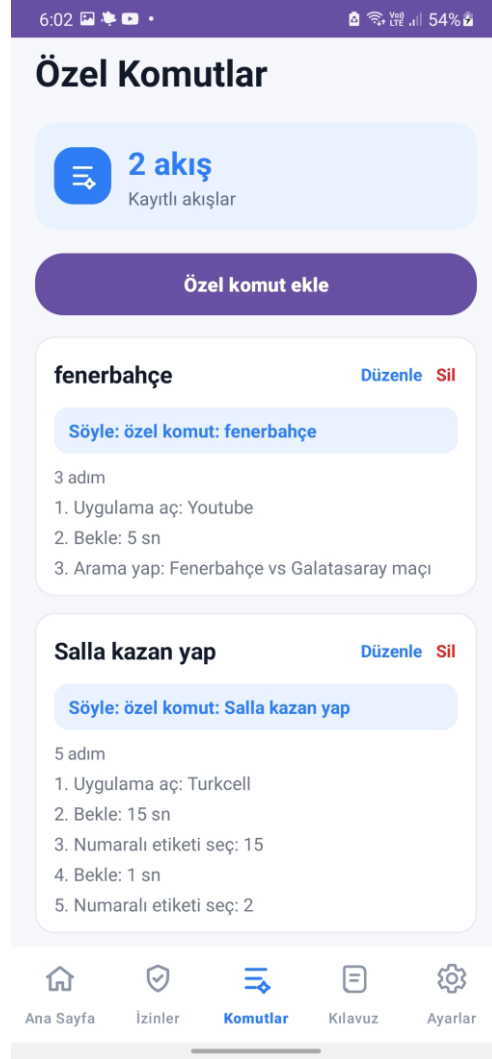
Uygulama izinleri, mikrofon, rehber, telefon araması yapma, telefon durumu, arama yanıtlama ve kamera erişimlerini kapsamaktadır. Gelişmiş erişim bölümünde ise erişilebilirlik servisi, diğer uygulamaların üzerinde pencere gösterme, ses modu yönetimi ve sistem ayarlarını değiştirme izinleri bulunmaktadır. Kullanıcı ilgili izin satırını seçerek Android izin penceresine veya gerekli sistem ayarı ekranına yönlendirilmektedir. Bu ekran Şekil 21’de gösterilmektedir.



Şekil 21. Uygulama içerisinde izin yönetme sayfası

5.4.3. ÖZEL KOMUTLAR EKRANI

Bu ekran, kullanıcının birden fazla işlemten oluşan sesli komut akışları oluşturmasını ve yönetmesini sağlamaktadır. Bu ekran üzerinden yeni bir özel komut eklenebilmekte, mevcut komutlar düzenlenebilmekte veya silinebilmektedir. Kayıtlı özel komutlar cihaz ve seçili dil bilgisiyle ilişkilendirilerek backend aracılığıyla PostgreSQL veritabanında saklanmaktadır. Bu ekran Şekil 22’de gösterilmektedir.



Şekil 22. Uygulama içerisinde özel komut işleme sayfası

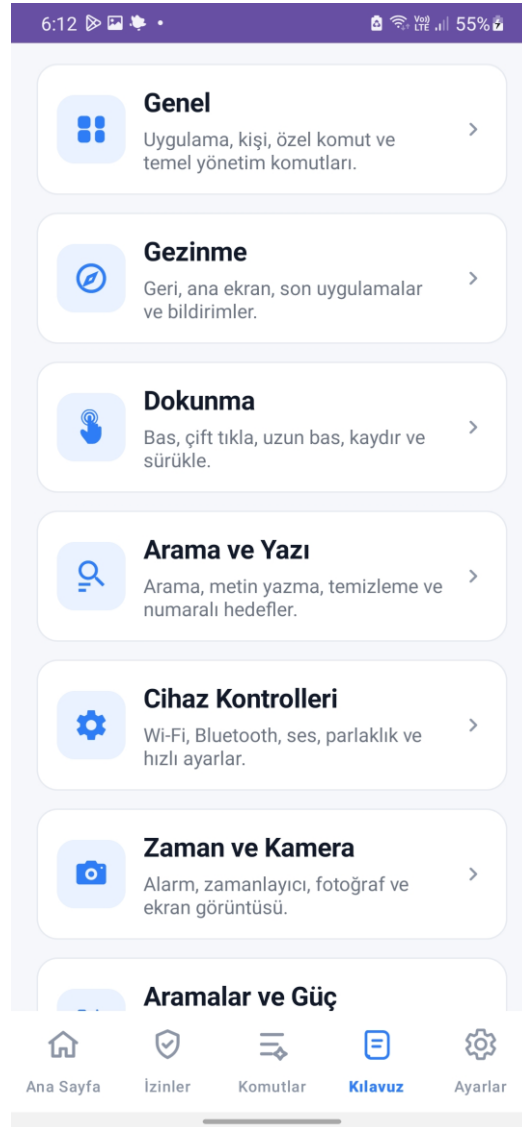
Bir özel komut; uygulama açma, uygulama içerisinde arama yapma, arayüz ögesine tıklama ve belirli bir süre bekleme adımlarından oluşabilmektedir. Kullanıcı özel komuta bir ad verdikten sonra akış adımlarını çalışma sırasına göre belirlemektedir. Kaydedilen akış, özel komutun adı sesli söylendiğinde backend tarafından bulunmakta ve adımlar Android uygulamasında sıralı biçimde yürütülmektedir.

Özel komut akışının yürütülmesi sırasında herhangi bir adımın başarısız olması durumunda sonraki adımlar çalıştırılmadan akış sonlandırılmaktadır. Kullanıcı, akış tamamlanmadan önce özel komutu istediği anda iptal edebilmektedir. Akış devam ederken yeni görev komutları işleme alınmamakta; yalnızca özel komutu iptal etme ve asistanı durdurma komutları kabul edilmektedir.

5.4.4. SESLİ KOMUT KILAVUZU EKRANI

Şekil 23'te gösterilen sesli komut kılavuz ekranı, sistem tarafında desteklenen komutların kullanıcı tarafından incelenmesini sağlamaktadır. Komutlar; Genel, Gezinme, Dokunma, Arama ve Yazı, Cihaz Kontrolleri, Zaman ve Kamera, Aramalar ve Güç olmak üzere görevlerine göre kategorilere ayrılmıştır.

Kullanıcı bir kategori seçtiğinde ilgili komutların açıklamalarına, kullanım biçimlerine ve söyleyebileceği örnek ifadelerle erişebilmektedir. Kılavuz içeriği uygulamada seçilen Türkçe, İngilizce veya Arapça dile göre gösterilmektedir. Böylece kullanıcı, desteklenen komutları ve gerekli ifade yapılarını doğrudan uygulama içerisinde öğrenebilmektedir.



Şekil 23. Uygulama içerisinde kullanıcı yardım kılavuzu sayfası

5.4.5. AYARLAR EKRANI

Şekil 24'te gösterilen ayarlar ekranı, uygulamanın dil ve görünüm tercihlerinin yapılandırılmasını sağlamaktadır. Dil ayarı üzerinden Türkçe, İngilizce veya Arapça seçilebilmektedir. Seçilen dil; uygulama arayüzünde, ses tanıma işlemlerinde, kılavuz içeriğinde ve backend'e gönderilen komut isteklerinde kullanılmaktadır.

Tema ayarı ile light/dark görünüm seçimi desteklenmektedir. Kullanıcının dil ve tema tercihleri cihazda saklanmakta ve uygulama yeniden açıldığında korunmaktadır. Ayrıca uygulamanın sürüm bilgisi de gösterilmektedir.



Şekil 24. Uygulama içerisinde ayarlar sayfası

5.5. UÇTAN UCA SİSTEM SONUCU

Proje sonucunda Android uygulaması, FastAPI backend servisi, XLM-RoBERTa tabanlı komut sınıflandırma modeli, rule engine, doğrulama katmanı, PostgreSQL veritabanı ve Redis cache bileşenleri birlikte çalışan uçtan uca bir sistem haline getirilmiştir. Kullanıcıdan alınan sesli komutlar Android tarafında metne dönüştürülmekte, backend tarafında intent ve parametre bilgilerine ayrıştırılmakta, doğrulandıktan sonra tekrar Android uygulamasına gönderilerek Erişilebilirlik Servisi ve sistem API'leri aracılığıyla çalıştırılmaktadır.

Geliştirilen sistem; uygulama açma, arayüz öğesine tıklama, kaydırma, metin yazma, metin temizleme, arama yapma, alarm/zamanlayıcı kurma, fotoğraf çekme, ekran görüntüsü alma, sistem ayarlarını değiştirme ve özel komut akışlarını çalıştırma gibi farklı görevleri desteklemektedir. Ayrıca İngilizce, Türkçe ve Arapça dilleri için komut işleme desteği sağlanmıştır. Bu yönüyle sistem, yalnızca tekil komutları değil, farklı Android ekranlarında değişen kullanıcı arayüzü yapılarını da dikkate alan daha esnek bir sesli kontrol yaklaşımı sunmaktadır.

Makine öğrenmesi modelinin sınıflandırma başarısının yanı sıra sistemin uçtan uca çalışma durumu da farklı komut senaryolarıyla değerlendirilmiştir. Bu değerlendirmelerde sesli komutun metne dönüştürülmesi, backend tarafından işlenmesi ve Android cihaz üzerinde beklenen işlemin gerçekleştirilmesi birlikte ele alınmıştır. Kullanılan test senaryoları ve gözlemlenen sonuçlar Tablo 6'da gösterilmiştir.

Tablo 6. Uçtan Uca İşlev Testleri

Komut kategorisi	Test senaryosu	Beklenen sonuç	Gerçekleşen durum
Uygulama yönetimi	“WhatsApp’ı aç”	WhatsApp uygulamasının açılması	Başarılı
Arayüz etkileşimi	“Paylaş butonuna bas”	Paylaş adaylarının bulunması veya seçilmesi	Başarılı
Metin işlemleri	“Merhaba yaz”	Metnin seçilen input alanına yazılması	Başarılı

Sistem ayarları	“Bluetooth’u aç”	Bluetooth etkinleştirilmesi	Başarılı
Zaman işlemleri	“Saat 7’ye alarm kur”	Alarm uygulamasında saat 07.00 alarmı oluşturulması	Başarılı
Özel komut	Kayıtlı çok adımlı komutun çalıştırılması	Adımların sırayla yürütülmesi	Başarılı

6. TARTIŞMA VE GELECEK ÇALIŞMALAR

Bu çalışmada geliştirilen Android AI Assistant sistemi, sesli komutların Android cihaz üzerinde işlevsel işlemlere dönüştürülmesini sağlayan bütünleşik bir yapı ortaya koymuştur. Sistem; SpeechRecognizer ile alınan komutları backend tarafında rule engine, XLM-RoBERTa tabanlı sınıflandırma modeli, doğrulama ve parametre zenginleştirme aşamalarından geçirerek Android tarafında Accessibility Service, gesture işlemleri, Android intentleri ve sistem API’leri aracılığıyla çalıştırmaktadır.

Sistemin güçlü yönlerinden biri, yalnızca makine öğrenmesi modeline bağlı kalmaması ve kural tabanlı çözümleme, doğrulama katmanı, uygulama kataloğu eşleştirmesi, dinamik kullanıcı arayüzü aday seçimi ve özel komut akışları gibi modüllerle desteklenmesidir. Bu yapı sayesinde hem deterministik komutlar hızlı şekilde çözümlenebilmekte hem de doğal dilde farklı biçimlerde söylenen komutlar model tarafından genellenebilmektedir.

Bununla birlikte sistemin bazı sınırlılıkları bulunmaktadır. Android SpeechRecognizer çıktıları ortam gürültüsü, arka planda çalan medya sesi, cihaz dili ve STT servisinin davranışına bağlı olarak değişebilmektedir. Ayrıca üçüncü taraf uygulamalar erişilebilirlik düğümlerini her zaman aynı açıklıkta sunmadığı için bazı ikon, buton veya input alanlarının algılanması uygulamadan uygulamaya farklılık gösterebilmektedir. Bu nedenle sistemde aday gösterme, numaralandırma, grid, ikon aliasları ve yakın eşleşme gibi yardımcı yöntemler kullanılmıştır.

Gelecek çalışmalarda daha kararlı ve sürekli çalışan bir ses algılama mimarisi değerlendirilebilir. Özellikle AudioRecord ve VAD tabanlı bir yapı ile arka plan seslerinin daha kontrollü şekilde ayrıştırılması sistemin güvenilirliğini artırabilir. Bunun yanında veri setinin düşük örnekli sınıflar için genişletilmesi, farklı Android sürümleri ve cihazlarda daha kapsamlı test yapılması, dinamik UI eşleştirme algoritmalarının geliştirilmesi ve özel komut akışlarının daha kullanıcı dostu hale getirilmesi sistemin ileriye dönük geliştirilebilecek başlıca yönleri arasındadır.

7.EKLER

7.1. İŞ PAKETLERİ

7.1.1. İŞ PAKETLERİ TABLOSU VE ÇIKTILARI

Tablo 7’da iş paketleri, vermesi gereken çıktıyı, başarı kriteri ölçüsü ve projede etkisi ele alınmıştır.

Tablo 7. İş paketleri, çıktıları ve projede etkisi

ID	İş Paketleri	Çıktısı	Başarı Kriteri	Projede % etkisi
Ç1	İP1	Gereksinim analizi dokümanı	Tüm komut ve kategorilerin tanımlanması	%20
Ç2	İP2	Mobil uygulamanın arayüzü	Android uygulamasının çalışması ve UI öğelerinin çalışması	%15
Ç3	İP3	Ses tanıma ve komut sınıflandırma modülü	Komutların en az %85 doğruluk oranıyla sınıflandırılması, geliştirilen modelin FastAPI backend servisi üzerinde çalıştırılması ve Android uygulamasının modele API aracılığıyla erişebilmesi	%25
Ç4	İP4	Telefon kontrol modülleri	Uygulama açma, Inputlara yazı yazma, Arama, Scroll, Fotoğraf çekme vb. işlemlerin doğru bir şekilde çalışması	%30
Ç5	İP5	Test ve Rapor	Modül ve komutlarının testinin ardından son raporun teslim edilmesi	%10

7.1.2. İŞ PAKETLERİ ZAMANLAMA TABLOSU

Tablo 8’de yapılan iş paketlerinin yapıldığı zaman aralığıyla birlikte listelenmiştir.

Tablo 8. İş zamanlama planlaması

ID	İş Paketi	1. ay	2. ay	3. ay	4. ay	5. ay	6. ay
İP1	Analiz, Gereksinim Belirleme	x					
İP2	Android uygulamasının temel arayüzü ve altyapısı		x				
İP3	Ses tanıma ve komut işleme modülünün geliştirilmesi			x	x		
İP4	Telefon kontrol modülleri (arama, scroll vb.)				x	x	
İP5	Testler, iyileştirme ve rapor				x	x	x

7.2. PROJENİN ETKİSİ VE KATKISI

7.2.1. SÜRDÜRÜLEBİLİR KALKINMA AMAÇLARI

Tablo 9. Sürdürülebilir Kalkınma Amaçları

1 	Yoksulluğa Son	10 	Eşitsizliklerin Azaltılması
2 	Açlığa Son	11 	Sürdürülebilir Şehirler ve Topluluklar
3 	Sağlık ve Kaliteli Yaşam	12 	Sorumlu Üretim ve Tüketim
4 	Nitelikli Eğitim	13 	İklim Eylemi
5 	Toplumsal Cinsiyet Eşitliği	14 	Sudaki Yaşam
6 	Temiz Su ve Sanitasyon	15 	Karasal Yaşam
7 	Erişilebilir ve Temiz Enerji	16 	Barış, Adalet ve Güçlü Kurumlar
8 	İnsana Yakışır İş ve Ekonomik Büyüme	17 	Amaçlar için Ortaklıklar
9 	Sanayi, Yenilikçilik ve Altyapı		

7.2.2. SKA AÇIKLAMASI

Proje, hareket kısıtı bulunan veya elleriyle dokunmatik ekranı kullanmakta güçlük yaşayan bireylerin Android cihazları sesli komutlarla kontrol edebilmesini desteklemektedir. Bu yönüyle dijital hizmetlere erişimde karşılaşılan fiziksel etkileşim engellerini azaltmayı hedefleyerek **SKA 10** ve **Hedef 10.2** ile ilişkilendirilmektedir. Ayrıca yapay zekâ, çok dilli komut işleme ve Android erişilebilirlik teknolojilerini bir araya getiren yenilikçi bir yazılım prototipi olması nedeniyle **SKA 9** ve **Hedef 9.5** ile ikincil düzeyde ilişkilidir.

7.2.3. SKA ALT MADDELERİ

Hedef 10.2: 2030 yılına kadar yaş, cinsiyet, engellilik, ırk, etnik köken, menşe, din, ekonomik durum veya diğer statülerine bakılmaksızın herkesin sosyal, ekonomik ve siyasi kapsayıcılığını güçlendirmek ve desteklemek. Proje, hareket kısıtı bulunan bireylerin Android cihazlarla sesli komutlar aracılığıyla etkileşim kurmasını desteklemesi nedeniyle bu hedefle ilişkilendirilmektedir.

Hedef 9.5: Bilimsel araştırmaların geliştirilmesi, teknolojik yetkinliklerin yükseltilmesi ve inovasyonun teşvik edilmesi. Proje; yapay zekâ, çok dilli komut işleme ve Android erişilebilirlik teknolojilerini bir araya getiren deneysel bir yazılım sistemi geliştirmesi nedeniyle bu hedefle ikincil düzeyde ilişkilendirilmektedir.

8.KAYNAKLAR

- Accessibility* | *Apple Developer Documentation*. (n.d.). Retrieved January 16, 2026, from <https://developer.apple.com/documentation/accessibility>
- AccessibilityService* | *API reference* | *Android Developers*. (n.d.). Retrieved January 16, 2026, from <https://developer.android.com/reference/android/accessibilityservice/AccessibilityService>
- Android Open Source Project*. (n.d.). Retrieved January 16, 2026, from <https://source.android.com/>
- App Sandbox* | *Apple Developer Documentation*. (n.d.). Retrieved June 18, 2026, from <https://developer.apple.com/documentation/security/app-sandbox>
- Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised Cross-lingual Representation Learning at Scale. *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 8440–8451. <https://doi.org/10.18653/V1/2020.ACL-MAIN.747>
- Download Android Studio & App Tools* - *Android Developers*. (n.d.). Retrieved June 29, 2026, from <https://developer.android.com/studio>
- FastAPI* - *FastAPI*. (n.d.). Retrieved June 18, 2026, from <https://fastapi.tiangolo.com/>
- Mobile Operating System Market Share Worldwide* | *Statcounter Global Stats*. (n.d.). Retrieved June 29, 2026, from <https://gs.statcounter.com/os-market-share/mobile/worldwide/>
- PostgreSQL: Documentation: 18: 1. What Is PostgreSQL?* (n.d.). Retrieved June 18, 2026, from <https://www.postgresql.org/docs/current/intro-what-is.html>
- Quick starts* | *Docs*. (n.d.). Retrieved June 18, 2026, from <https://redis.io/docs/latest/develop/get-started/>
- Security of runtime process in iOS, iPadOS, and visionOS* - *Apple Support*. (n.d.). Retrieved January 16, 2026, from <https://support.apple.com/guide/security/security-of-runtime-process-sec15bfe098e/web>
- Speech to Text Guide: Convert Audio to Text 3x Faster (2025)*. (n.d.). Retrieved January 17, 2026, from <https://voicetonotes.ai/blog/speech-to-text-guide/>
- SpeechRecognizer* | *API reference* | *Android Developers*. (n.d.). Retrieved January 17, 2026, from <https://developer.android.com/reference/android/speech/SpeechRecognizer>
- Use Voice Access to control your Android device with your voice*. (n.d.). Retrieved January 16, 2026, from <https://blog.google/company-news/outreach-and-initiatives/accessibility/voice-access-updates/>
- Voice Access* - *Apps on Google Play*. (n.d.). Retrieved January 16, 2026, from <https://play.google.com/store/apps/details?id=com.google.android.apps.accessibility.voice.access&hl=en>
- XLNet* · *Hugging Face*. (n.d.). Retrieved June 18, 2026, from https://huggingface.co/docs/transformers/model_doc/xlnet
- Zhong, Y., Raman, T. V., Burkhardt, C., Biadsy, F., & Bigham, J. P. (2014). JustSpeak: Enabling universal voice control on android. *W4A 2014 - 11th Web for All Conference*. <https://doi.org/10.1145/2596695.2596720>

9.ÖZGEÇMİŞ

Ben Muhammed Chreiki, İstanbul Topkapı Üniversitesi Yazılım Mühendisliği Bölümü öğrencisiyim. Yazılıma duyduğum ilgi ve öğrenme isteği doğrultusunda özellikle yapay zekâ, makine öğrenmesi ve mobil uygulama geliştirme alanlarında çalışmalar yürütüyorum. Edindiğim teorik bilgileri gerçek projelere aktararak problem çözme ve yazılım geliştirme becerilerimi sürekli geliştirmeye önem veriyorum.

Kariyer hedefim; frontend, backend, veritabanı ve yapay zekâ teknolojilerine hâkim, nitelikli bir Full-Stack Yazılım Geliştiricisi olmaktır. Çocukluğumdan beri kurduğum mühendis olma hedefini gerçekleştirmek için disiplinli, kararlı ve sürekli öğrenmeye dayalı bir çalışma anlayışıyla ilerliyorum.

LinkedIn Profilim: <https://www.linkedin.com/in/se-muhammed-chreiki/>

GitHub: <https://github.com/SE-ABOSALIM>